Application Note

使用 GD30BM2016 的移植事项

摘要

GD30BM2016 是一款高度集成的高精度电池监控器和保护器，该器件包含一个高精度监控系统和一个高度可配置的保护子系统，并支持自主式或主机控制型电池平衡。主控器件在 GD30BM2016 上电时，根据系统要求对该器件进行初始化流程配置，读取状态和数据结果。

GD30BM2016 可以做到与 BQ76952 完全 PIN2PIN 兼容，外围电路兼容设计，但寄存器、引脚定义等与 BQ76952 有略微差异。本文主要介绍了使用 GD30BM2016 时的软件移植流程，目的是配合用户更快的实现系统级应用。

目录

1 器件连接注意事项 3
2 GD30BM2016 与 BQ76952 差异 4
2.1 硬件差异 4
2.1.1 引脚对比 4
2.1.2 未使用引脚的处理 4
2.2 性能和功能对比 5
2.2.1 均衡差异 5
2.2.2 DSG 和 CHG 差异 6
2.2.3 采样数据处理 7
2.2.4 VC0 和 VC1 低串 8
2.2.5 功耗对比 8
2.3 寄存器差异 9
2.3.1 寄存器地址的差异 9
2.3.2 电芯串数配置和电压读取 9
2.3.3 内部 LDO 的使能和电压选择 10
2.3.4 多功能引脚的配置 10
2.3.5 NTC 测量引脚的配置 13
2.3.6 保护阈值与延迟的 STEP 的差异 14
2.3.7 CHG_PUMP_CONTROL 配置 14
2.3.8 电流寄存器差异 14
2.4 软件配置差异 16
2.4.1 通信差异 16
2.4.2 电压串数 16
2.4.3 电流增益设置和读取 16
2.4.4 寄存器和命令解密 17
3 BM2016 常用功能注意事项 17
3.1 BM2016 上电时序 17
3.1.1 硬件复位芯片 17
3.1.2 软件复位芯片 19
3.2 BM2016 配置模式 20

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

3.3 电压配置和寄存器 21
3.4 电流配置和寄存器 22
3.5 温度配置和读取 24
3.6 FET 控制 27
3.7 均衡使用 31
3.7.1 自动均衡 31
3.7.2 手动均衡 33
3.8 保护功能 35
3.9 中断 ALERT 处理 36
3.10 WatchDog 功能使用 38
4 初始化流程 41
4.1 初始化配置示例 41
4.1.1 CONFIG_UPDATE 模式 41
4.1.2 保护寄存器配置 42
4.1.3 引脚复用功能 43
4.1.4 ADC 配置 44
4.1.5 系统寄存器配置 44
4.1.6 均衡寄存器配置 44
4.1.7 初始化配置生效 45
4.2 采样数据和寄存器状态读取 45
4.3 采样数据转换计算 46
5 通信函数封装 47
5.1 直接命令函数 47
5.2 不带数据子命令函数 48
5.3 带数据子命令函数 48
5.4 RAM 寄存器读写函数 51
5.5 CRC 校验 53
5.6 CheckSum 校验 54
6 命令和寄存器宏定义 55
6.1 GD30BM2016 头文件 55
7 历史版本 68

Copyright © 2024, GigaDevice Semiconductor Inc.
All Rights Reserved.

1 器件连接注意事项

用户在使用 GD30BM2016 时，连接和配置必须按照下述说明使用，如果不按要求的作业，有可能会损坏保护板的元器件，从而导致器件不能保护电芯，造成严重的后果。

连接步骤

1. 先连接电池组的动力线负极 BAT-到 PCB 板上的 BAT-；
2. 接着连接电池组的采样排线到 PCB 板上的电池采样口；
3. 最后连接电池组的动力线正极 BAT+到 PCB 板上的 BAT+；
4. 所有连接线安装好，再激活电池保护板开机，确认电池总电压与保护板输出电压相等。

断开保护板的步骤

1. 先将 PACK+至 PACK-的负载移除，然后通过指令关闭充放电 MOSFET；
2. 接着断开电池组正极的 BAT+连接线；
3. 然后拔下电池组的采样排线；
4. 最后断开电池组负极的 BAT-连接线。

未使用 CELL 的连接

对于在设计中未使用的 CELL 输入，应该与之相邻有效 CELL 输入直接短接，不可以浮空并且将 vcell_mode 寄存器中未使用 cellx bit 设置为 0。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

2 GD30BM2016 与 BQ76952 差异

GD30BM2016 和 BQ76952 硬件上可以做到完全的 pin to pin 兼容，标准的推荐电路，无需做任何 BOM 的更改。但在引脚定义、功能和使用方法上略有差异，该文档整理了硬件、性能、寄存器等差异点，具体如下：

2.1 硬件差异

BM2016 高压部分芯片的面积大于 BQ76952，高压面积增大意味这更大的功率承受能力，对于外部浪涌冲击具有更大的耐受特性。

2.1.1 引脚对比

引脚号	GD30BM2016	BQ76952	备注
19	OTP	NC	当用户需要烧写 OTP 时候，需要将该引脚接一个 2.2uF 电容，耐压 16V。如果不涉及到 OTP 烧写，忽略此功能
28	SPI_MOSI	HDQ	GD30BM2016 不支持 HDQ 的通信方式，该引脚默认名字是 SPI_MOSI，在 I2C 模式下该引脚电平可以选择设备地址，浮空或者拉低时设备地址为 0x20，SPI_MOSI 拉高时设备地址为 0x22, 区别与 BQ76952 上电只支持 0x10，其它功能相同均可以作为热敏电阻输入引脚。
33	RST_SHUT	RST_SHUT	GD30BM2016 该引脚内部下拉为 220k ohm, BQ76952 内部下拉为 1M ohm 电阻，用户如果在外围电路接入上拉电阻，需要注意上拉电阻的取值以便于上拉电平的识别。
18/20	SRP/SRN	SRP/SRN	该引脚为模拟输入引脚，用以采样电流。 GD30BM2016 引脚可支持的共模电压最大为 3.2V，BQ76952 为 1V，当引脚电压超过支持的最大共模电压，芯片会被强行复位。更大的共模电压可以在 BAT-和 P- 大电流瞬间断开时，留足够的时间触发 SCD 保护避免 SRP -SPN 之间的采样电阻烧断。BM2016 SRP-SRN 的差模电压范围也大于 BQ76952, 意味着可以承受更大的瞬间短路电流，避免在瞬间大电流短路时复位 AFE。
31	DCHG	DCHG	GD30BM2016 引脚多了一个复用功能，可以复用为看门狗超时事件的输出信号，用户可以监测该信号，配合主控灵活的控制 AFE 的复位。
48	VC16	VC16	BQ76952 在顶端均衡时，由于从 VC16 取电因此需要更小的 RC 避免对 VC16 电压的影响，一般 TI 建议客户选择 20ohm 的 VC16 输入电阻. GD30BM2016 在均衡期间 VC1-VC16 的电压不受影响，VC16 的输入电阻支持 20ohm，100ohm 等，更大的输入电阻可以提高该引脚的抗浪涌能力。
25	Alert	Alert	BM2016 增加芯片唤醒 wake 中断源，在大电流唤醒 AFE 时将中断信号输出给 MCU，作为 MCU 的唤醒标志。

2.1.2 未使用引脚的处理

BQ76952 和 BM2016 在未使用引脚的处理上有微小差异，下表给出 BM2016 未使用引脚的处理方法。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

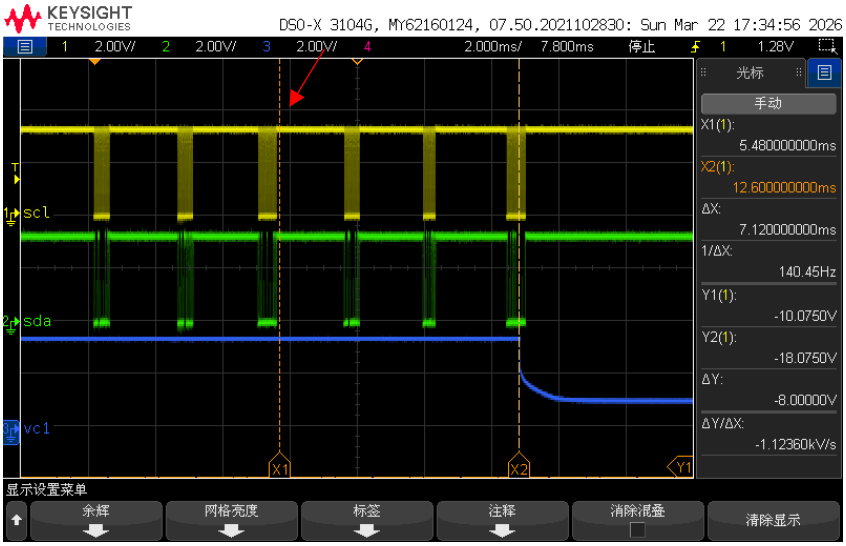
引脚	名称	建议
1~16、48	VC0~VC16	电芯输入 1、2 和 16 应始终连接到实际电芯，其中电芯连接在 VC1 和 VC0 之间、VC2 和 VC1 之间以及 VC16和 VC15 之间。VC0 应通过 PCB 上的电阻器和电容器连接到引脚 17 (VSS)。将未使用的 VC 引脚直接短接至相邻的 VC 引脚。所有 VC 引脚都应连接到相邻的 VC引脚、实际电芯（通过 R 和 C）或电池组互连电阻（通过 R 和 C）。
18、20	SRP、SRN	如果未使用，这些引脚应该连接到引脚 17 (VSS)。
19、44	NC	19 脚 OTP 烧写用，需悬空；44 脚悬空。
21、23、25、28、29、30、31、32	TS1、TS3、ALERT、SPI_MOSI、CFETOFF、DFETOFF、DCHG、DDSG	如不使用，这些引脚既可以保持悬空状态，也可以连接到引脚 17 (VSS)。
22	TS2	如果器件要进入 SHUTDOWN 模式，则 TS2 引脚应悬空。如果该引脚用于芯片唤醒，可以连接外部三极管用于 MCU 控制。
33	RST_SHUT	如果未使用，此引脚应该连接到引脚 17 (VSS)。
34，35	REG1、REG2	如不使用，这些引脚既可以保持悬空状态或者通过 0.1uF 电容接引脚 17 (VSS)。
36	REGIN	如果未使用，此引脚应该通过 0.1uF 电容连接到引脚 17 (VSS)。
37	BREG	如果未使用，此引脚应该通过 0.1uF 电容连接到引脚 17 (VSS)。
38	FUSE	如不使用，该引脚既可以保持悬空状态，也可以连接到引脚 17 (VSS)。
39	PDSG	如果未使用，该引脚应该保持悬空。
40	PCHG	如果未使用，该引脚应该保持悬空。
41	LD	如果不使用 DSG 驱动器，则可以通过串联电阻器将该引脚连接到 PACK+ 连接器，或者连接到引脚 17 (VSS)。
43	DSG	如果未使用，该引脚应该保持悬空。
45	CHG	如果未使用，该引脚应该保持悬空。
46	CP1	如果未使用，此引脚应该连接到引脚 47 (BAT)。注意：此时启用电荷泵，将额外消耗 800uA 的电流。

2.2 性能和功能对比

2.2.1 均衡差异

- BM2016 在均衡时，均衡时序和采样时序完全数字控制错开，因此 VC1-VC16 的均衡不会影响采样结果和精度。BQ76952 的均衡从 VC16 取电，均衡和采样存在并行的工况，当均衡较多时，由于 VC16 的 RC 输入电阻，会造成测量结果偏低。
- 当选择的电池串数低于 16 串时，BQ76952 每个 cell 的占空比不同，也就是说平均均衡速度会有差异，接入的电池串数越少该现象越明显。BM2016 每个 cell 的均衡不会把不选择的 cell 控制去除，每个 cell 的均衡占空比完全一致，均衡占空比只跟 loop_slow 有关，loop_slow 越大占空比越大。
- BM2016 采用数字状态机控制均衡的开关，当手动均衡命令下发后，BM2016 每次都稳定的 2ms 后打开对应 cell 的均衡。BQ76952 采用软核的方式控制均衡的打开，手动均衡命令下发到生效有时需要 1S 才能打开均衡开关。

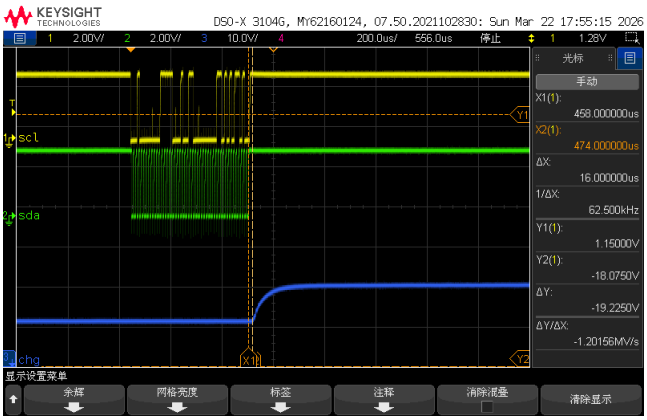
Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

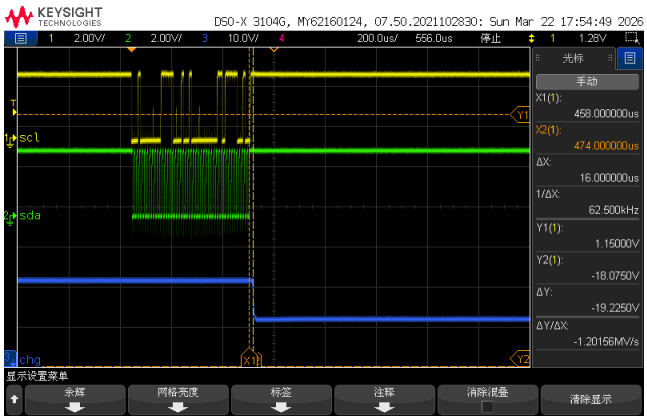


- GD30BM2016 的内部均衡能力为 80mA，BQ76952 的内部均衡能力为 50mA。

2.2.2 DSG 和 CHG 差异

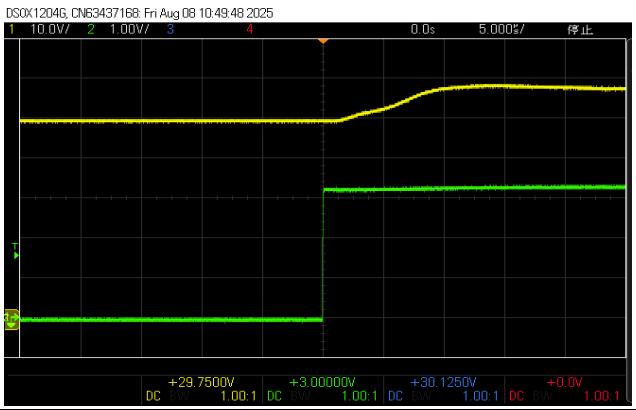
- BM2016 CHG SOURCE 和 SINK 等效阻抗是 380ohm，DSG SOURCE 和 SINK 等效阻抗是 800ohm，BQ76952 内部等效阻抗是 2000ohm，这意味着 BM2016 开关驱动电流更大，驱动速度更快。
- BM2016 DSG 和 CHG 的控制都是跟随硬件数字状态机，用户再发送打开 MOS 命令或者关闭 MOS 命令后，约 us 后，BM2016 会操作对应的 MOS。但 BQ76952 由于是软核，需要更多的时间控制 MOS 开关且时间不固定。

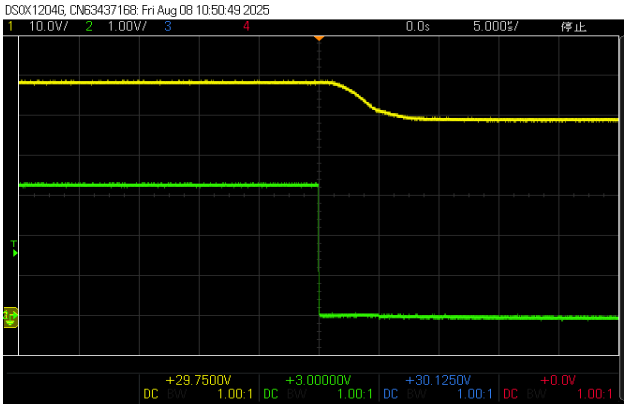




- DSG 和 DCHG 作为驱动状态的输出信号，BM2016 的 MOS 动作和驱动状态输出信号电平基本同步，MCU 检测电平可以迅速知道 MOS 动作的结果。BQ76952 需要更多的时间才能把 MOS 状态输出到引脚上。

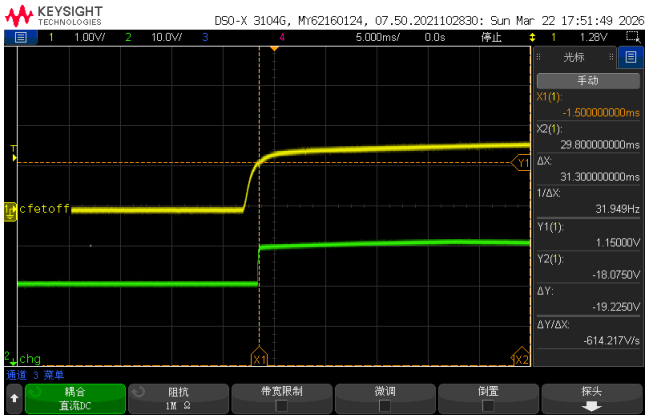
  CH1：CHG 信号　CH2：DCHG 信号

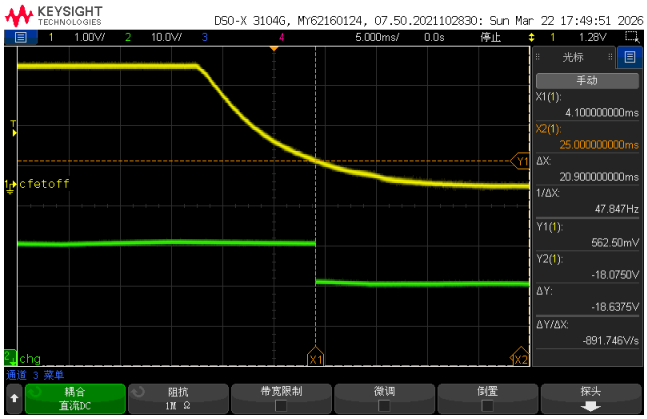




Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

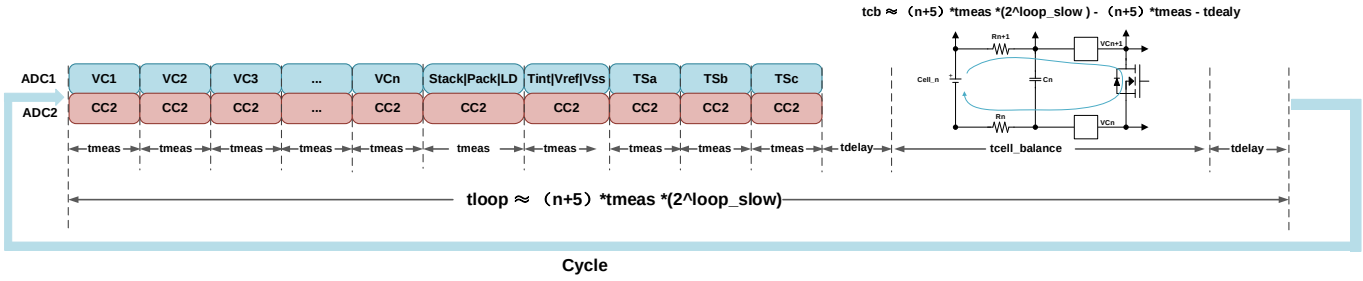
- 同理 CFETOFF 信号、DFETOFF 信号也可以在 100us 级别控制 MOS 的打开和关断，对于高低边关联或者多串 AFE 级联控制，BM2016 实时性更高更稳定。





2.2.3 采样数据处理

- BM2016 支持不同 CELLX 输入电阻的补偿，当前 BM2016 在工厂 ATE 阶段使用 20ohm 的输入电阻做校准，如果客户使用 100ohm 电阻（推荐值），可以直接回读 Cellx gain，将原始的 Cellx gain * 1.0013 写入到 CellX gain 寄存器中。如果用户为了提高浪涌能力，选择 1000ohm 的输入电阻，直接回读 Cellx gain，将原始的 Cellx gain * 1.0049 写入到 CellX gain 寄存器中
- BM2016 和 BQ76952 的电压电流都为双 ADC 同步采样，但采样速度不同，BQ76952 的 VADC 和 CADC 最快采样速度为 3mS。BM2016 电压采集 VADC 和 CADC 的时钟为 1KHZ，采样周期默认为 1ms，pack/Stack/LD 为一组轮循采样，每三个温度引脚为一组轮循采样。除非系统对获取电压速度要求很高即对采样速度要求很高，一般将以将 VADC 设置为 4ms，这样可以提升采样精度。



- CC1 Current 是 100 个 CC2 Current 的平均值，CC1 Current 用来判断芯片是否进入 sleep 状态。 相比于 BQ76952，BM2016 增加了 VADC 和 CADC 时钟分频配置寄存器：ADC Clk Cfg Register(0x8049)。增大 CADC 的时钟分频等同于增大 CC1 CURRENT 的刷新周期，如果用户配置了 sleep 模式下关闭充电 MOS，更长 CC1 CURRENT 的刷新周期可以更晚的进入 sleep，充电 MOS 关闭的更晚一些。对于一些应用例如三相电机停机提供了更长的放电续流时间。

DC Clk Cfg Register | Address : 0x00008049				
Field	**Bits**	**Default**	**Type**	**Field Description**
cadc_clk_sel1	[7: 6]	0x0	RW	Sleep/DeepSleep mode：curr adc clk divider 0: 1 Frequency division 1: 2 Frequency division 2: 4 Frequency division 3: 8 Frequency division
cadc_clk_sel0	[5: 4]	0x0	RW	Normal mode：curr adc clk divider 0: 1 Frequency division

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

DC Clk Cfg Register | Address : 0x00008049				
Field	**Bits**	**Default**	**Type**	**Field Description**
				1: 2 Frequency division 2: 8 Frequency division 3: 24 Frequency division
vadc_clk_sel1	[3: 2]	0x0	RW	Sleep/DeepSleep mode：volt adc clk divider 0: 1 Frequency division 1: 2 Frequency division 2: 4 Frequency division 3: reserve writing '1' is prohibited
vadc_clk_sel0	[1: 0]	0x0	RW	Normal mode：volt adc clk divider 0: 1 Frequency division 1: 2 Frequency division 2: 4 Frequency division 3: reserve writing '1' is prohibited

- BM2016 和 BQ76952 都支持库仑计，BQ76952 仅支持在 normal 模式下，BM2016 在 normal 和 sleep 模式均支持，这意味着在小电流期间 BM2016 一直对电流做积分，用户可以调用带数据的子命令获取库仑计结果。

DAStatus6_0 Register Address: 0x120

Field	Bits	Default	Type	Field Description
accum_charge	[31:0]	0x0	RO	Reports the integer portion of accumulated passed charge in userAmp-hours.

DAStatus6_1 Register Address: 0x124

Field	Bits	Default	Type	Field Description
accum_charge_fraction	[31:0]	0x0	RO	Reports the fractional portion of accumulated passed charge. This is initialized to 0.5 userAh to facilitate appropriate rounding of the integer portion.

- BQ76952 和 BM2016 都支持内部上拉电阻，可以通过直接命令读取温度值。BM2016 NTC 系数只有 COEFF_A 一组，而 BQ76952 有两组系数。在高低温下，BM2016 需要做二次修正补偿误差。获取用户使用的热敏电阻型号或者温度和 NTC 阻值的曲线，GD 提供相关 COEFF_A 系数和二次修正系数。用户也可以采用查表法，读热敏电阻输入引脚的电压，通过查表计算对应的阻值获取温度，查表法下支持使用内部温度上拉电阻，也支持断开内部上拉电阻选择外部上拉电阻。

- NTC 内部采用了 18K 上拉电阻，NTC 的输入电容不要太大，建议不要超过 1nF,否则 RC 充电速度太慢会造成电压还没有建立稳定就采样，采样值偏低。由于 BM2016 的采样速度更快，更需要注意滤波的电容值。

2.2.4 VC0 和 VC1 低串

BM2016 和 BQ76952 的 VC0 对地都不是高压管，均需要并联稳压管到 VSS，避免充放电过程 VC0 浮动电压损坏 VC0。BM2016 可以-0.5V 到 1.2V 范围内的共模电压，CELL1 电压精度无损失。BQ76952 在-0.5V 到 0.5V 共模电压范围内，CELL 1 电压精度无损失。BM2016 相比与 BQ76952 具有更大的模拟电压抑制，不会对于 CELL1 电压精度有影响。

2.2.5 功耗对比

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

BQ76952 在正常模式下运行功耗约 400uA, BM2016 运行功耗约 960uA，主要差别 1 是 BM2016 的采样速度是 BQ76952 的三倍，2 是电荷泵驱动提升造成。其他模式功耗差不多，用户可以配置 0x8000 寄存器中的 loop_slow 降低 normal 下的功耗。用户也可以通过配置采样周期、chgpump 等进一步降低芯片总功耗。

模式	BQ76952 功耗	BM2016 功耗
Normal	400uA	960uA
Sleep	46uA	60uA
DeepSleep	11uA	20uA
Shutdown	3uA	5uA

2.3 寄存器差异

2.3.1 寄存器地址的差异

GD30BM2016 和 BQ76952 的寄存器命名规则基本相同，但是偏移地址不同，具体地址可以参考 *GD30BM2016 Register Maps*。下表是 GD30BM2016 的基地址，用户也可以不用查看寄存器差异的细节，直接使用 GD30BM2016 头文件，GD30BM2016 地址在头文件中做了宏定义地址映射，参考 *GD30BM2016 头文件*。寄存器不同的部分，会单独说明。

模块	基地址	描述
indircmd_with_data	0x00000000	带有数据的间接子命令，通过间接子命令可以获取电压 ADC 和电流 ADC 的原始 count 结果，均衡时间的值，作为辅助数据使用。与寄存器操作方式相同，需要进入 configure update 即可操作。
indircmd_only	0x00000300	不带数据的间接子命令，主要用于控制使用。可以用于复位，模式切换，GPIO 的控制，MOSFET 的控制等。
Dircmd	0x00000400	直接命令主要 cell 电压，辅助电压，电流和温度采集，以及工作状态读取。使用户初始化之后，获取芯片数据和状态最为重要的部分。
Sysc	0x00008000	系统核心参数设置，如工作模式，通信方式，LDO 输出，MOSFET 控制，OTP 相关参数配置。
Pad	0x00008200	多功能引脚设置，多功能引脚复用相关参数设置，如 NTC 的配置，GPIO，多功能引脚复用等。
Adc	0x00008400	芯片测量相关校准参数，大部分工厂已经做好校准，部分需要用户根据外围来修改，用户也可以做二次优化提升测量精度。
Protect	0x00008800	保护相关的参数，所有一次和二次保护相关参数设置。
Cb	0x00008c00	cell balance，均衡和负载检测相关的参数设置。

2.3.2 电芯串数配置和电压读取

GD30BM2016 的 16 个 cell 是从 CELL1-CELL9, CELL10-CELL16，其中 bit8 是空出来保留位，固定设置为 0。例如设置 14 串模式，外部连接将 VC15, VC14 以及 VC13 短接 vcell_mode 设置为 0x9EFF。

Register: VCELL_MODE | Address: 0x00008808				
Field	**Bits**	**Default**	**Type**	**Field Description**

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

vcell_mode	[15: 0]	0x0	RW	[15] : Cell 16 Mode [14] : Cell 15 Mode [13] : Cell 14 Mode [12] : Cell 13 Mode [11] : Cell 12 Mode [10] : Cell 11 Mode [9] : Cell 10 Mode [8] : RSV [7] : Cell 9 Mode [6] : Cell 8 Mode [5] : Cell 7 Mode [4] : Cell 6 Mode [3] : Cell 5 Mode [2] : Cell 4 Mode [1] : Cell 3 Mode [0] : Cell 2 Mode Cell 1 is fixed in use each bit 1 : means this cell is in use each bit 0 : means only cell1 use, other cells is not in use

配置完 Vcell mode 生效后，通过直接子命令方式获取电芯电压，这里和 BQ76952 命令码不同，下表是 16 个 cell 电压的直接子命令码。

Cell Volt 1	Cell Volt 2	Cell Volt 3	Cell Volt 4	Cell Volt 5	Cell Volt 6	Cell Volt 7	Cell Volt 8
0x14	0x16	0x18	0x1A	0x1C	0x1E	0x20	0x20
Cell Volt 9	Cell Volt 10	Cell Volt 11	Cell Volt 12	Cell Volt 13	Cell Volt 14	Cell Volt 15	Cell Volt 16
0x24	0x28	0x2A	0X2C	0X2E	0X30	0X32	0X3C

2.3.3 内部 LDO 的使能和电压选择

REG12_CONFIG 寄存器，定义 REG1 和 REG2 输出电压时候，请参考下表，与 BQ76952 存在差异。

Register: REG12_CONFIG | Address: 0x00008002				
Field	**Bits**	**Default**	**Type**	**Field Description**
reg2v	[7: 5]	0x0	RWHW	Selects voltage level for REG2 This setting should not be changed while REG2 is enabled. 0= 3.3V, 1= 5V, 2= 3V, 3 = 2.5 V, 4-7 = 1.8V
reg2_en	[4: 4]	0x0	RWHW	Configure default state for REG2 output. This setting is reapplied when initializing after reset or DEEPSLEEP mode. 0 = REG2 Disabled 1 = REG2 Enabled
reg1v	[3: 1]	0x0	RWHW	Selects voltage level for REG1 This setting should not be changed while REG1 is enabled. 0= 3.3V, 1= 5V, 2= 3V, 3 = 2.5 V, 4-7 = 1.8V
reg1_en	[0: 0]	0x1	RWHW	Configure default state for REG1 output. This setting is reapplied when initializing after reset or DEEPSLEEP mode. 0 = REG1 Disabled 1 = REG1 Enabled

2.3.4 多功能引脚的配置

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

涉及到 DDSG, DCHG, CFETOFF,DFETOFF, ALERT，SPI_MOSI 等引脚，选择驱动电压控制位 OPT[3]，选择 0 为输出驱动为 REG1,选择 1 输出驱动电平为 REG18，这里电平的配置与 TI 的 BQ76952 相反，如果配置的电平不合适，可能与 MCU 电平不匹配从而导致 sleep 模式下漏电。

以下以 ALERT 信号为例说明，但是针对其他复用引脚作为 GPIO 或者 ALT 功能时候都一样。

Register: ALERT_PIN_CONFIG | Address: 0x00008202				
Field	**Bits**	**Default**	**Type**	**Field Description**
alert_opt	[7: 2]	0x0	RW	这些位配置所选引脚功能的选项。 OPT[5]： --- 配置为 ALERT 时的极性（不影响 GPO 模式）--- 0 = 选择高电平有效 --- --- 1 = 选择低电平有效 OPT[4]： --- 0 = 数字输出(OEN=0) --- 1 = 数字不输出(OEN=1,HZ) OPT[3]： --- 0 = 输出高电平驱动使用 REG1 --- 1 = 输出高电平驱动使用 REG18 OPT[2]： --- 0 = 关闭数字输入(IE=0) --- 1 = 开启数字输入(IE=1) OPT[1]： --- 0 = 关闭数字输入弱上拉 --- 1 = 开启数字输入弱上拉 OPT[0]： --- 0 = 关闭数字输入弱下拉 --- 1 = 开启数字输入弱下拉 当为 ADCIN 或热敏电阻功能选择引脚时，将使用 OPT[5:0] 位，如下所示。 OPT[5:4]： --- 如果未使用，OPT[4]设置为 1. OPT[3]是否为 ADC 输入： 0 = 否 1 = 是 OPT[2]：是否为上拉电阻 Tsensor 输入： ---0 = 否 ---1 = 是 OPT[1]： --- 0 = 未用于 FET 温度保护 --- 1 = 用于 FET 温度保护 OPT[0]： --- 0 = 未用于电芯温度保护 --- 1 = 用于电芯温度保护
alert_pin_fxn	[1: 0]	0x0	RW	配置此引脚用于哪项功能。 0 =未使用 1 = 通用输出 2 = ALERT 3 = ADC 输入或热敏电阻

关于 DDSG 和 DCHG 作为 GPIO, 模拟输入或者作为 DSG 和 CHG 的影子信号的配置，与 BQ76952 也有一些不同：

- 如果要将 DDSG 和 DCHG 配置成 DSG 和 CHG 的影子信号，也就是随着高边 DSG 和 CHG 的变化，作为低压侧 DDSG 和 DCHG 要做同样的联动，需要将 DCHG_PIN_CONFIG 和 DDSG_PIN_CONFIG 寄存器设置为 0x02，此时电平和 MOS 状态是相同的。如果配置成 0x82 就是反电平。
- 如果要使能原有与 BQ76952 相同的 DDSG 和 DCHG 其他复用功能，不作为 DDSG 和 DCHG 的影子信号，需要将辅助控制寄存 0x8E2A 寄存器的 bit1 和 bit0 设置为 1。

例如：如果设置 DDSG 为 DSG 的影子信号，并且电平相同，设 DDSG_PIN_CONFIG(0x8208) 为 0x82。如果想设置 DDSG 的引脚作为 NTC 温度测量，上拉电阻，并且作为 MOSFET 温度保护。需要设置辅助寄存器 0x8E2A 的 bit1 为 1，并且配置 DDSG_PIN_CONFIG(0x8208) 为 0x5B。

Register: DCHG_PIN_CONFIG | Address: 0x00008207				
Field	**Bits**	**Default**	**Type**	**Field Description**
dchg_opt	[7: 2]	0x0	RW	这些位配置所选引脚功能的选项。 多功能引脚，可以是 CHG、DCHG、热敏电阻输入、通用 ADC 输入、通用

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

				数字输出、驱动输出; 当 0x8e2a[0]=0 时，DCHG 作为 CHG 的影子信号： OPT[5]: --- 0: 低边 CHG 功能，CHG 使能时，输出为低; ---1: 低边 CHG 功能，CHG 使能时，输出为高; 当 0x8e2a[0]=1 时，DCHG 作为常规 GPIO 映射： OPT[5]： --- 配置为 DCHG 时的极性（不影响 GPO 模式） --- --- 0 = 选择高电平有效 --- --- 1 = 选择低电平有效 OPT[4]： --- 0 = 数字输出(OEN=0) --- 1 = 数字不输出(OEN=1,HZ) OPT[3]： --- 0 = 输出高电平驱动使用 REG1 --- 1 = 输出高电平驱动使用 REG18 OPT[2]： --- 0 = 关闭数字输入(IE=0) --- 1 = 开启数字输入(IE=1) OPT[1]： --- 0 = 关闭数字输入弱上拉 --- 1 = 开启数字输入弱上拉 OPT[0]： --- 0 = 关闭数字输入弱下拉 --- 1 = 开启数字输入弱下拉 作为 ADCIN 或热敏电阻功能选择引脚时： OPT[5:4]： --- 如果未使用，OPT[4]设置为 1. OPT[3]是否为 ADC 输入： 0 = 否 1 = 是 OPT[2]：是否为上拉电阻 Tsensor 输入： ---0 = 否 ---1 = 是 OPT[1]： --- 0 = 未用于 FET 温度保护 --- 1 = 用于 FET 温度保护 OPT[0]： --- 0 = 未用于电芯温度保护 --- 1 = 用于电芯温度保护
dchg_pin_fxn	[1: 0]	0x0	RW	配置此引脚用于哪项功能。 0 = RSV 1 = 通用输出 2 = DCHG 3 = ADC 输入或热敏电阻

Register: DDSG_PIN_CONFIG | Address: 0x00008208				
Field	**Bits**	**Default**	**Type**	**Field Description**
ddsg_opt	[7: 2]	0x0	RW	多功能引脚，可以是 DSG、DDSG、热敏电阻输入、通用 ADC 输入、通用数字输出、驱动输出: 当 0x8e2a[1]=0 时,DDSG 作为 DSG 的影子信号： OPT[5]: --- 0: 低边 DSG 功能，DSG 使能时，输出为低; --- 1: 低边 DSG 功能，DSG 使能时，输出为高; 当 0x8e2a[1]=1 时，DDSG 作为常规 GPIO 映射： OPT[5]： --- 配置为 DDSG 时的极性（不影响 GPO 模式） --- --- 0 = 选择高电平有效 --- --- 1 = 选择低电平有效 OPT[4]： --- 0 = 数字输出(OEN=0) --- 1 = 数字不输出(OEN=1,HZ) OPT[3]： --- 0 = 输出高电平驱动使用 REG1 --- 1 = 输出高电平驱动使用 REG18 OPT[2]： --- 0 = 关闭数字输入(IE=0) --- 1 = 开启数字输入(IE=1) OPT[1]： --- 0 = 关闭数字输入弱上拉 --- 1 = 开启数字输入弱上拉 OPT[0]： --- 0 = 关闭数字输入弱下拉 --- 1 = 开启数字输入弱下拉 作为 ADCIN 或热敏电阻功能选择引脚时：

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

				OPT[5:4]： - 如果未使用，OPT[4]设置为 1. OPT[3]是否为 ADC 输入： 0 = 否 1 = 是 OPT[2]：是否为上拉电阻 Tsensor 输入： ---0 = 否 ---1 = 是 OPT[1]： --- 0 = 未用于 FET 温度保护 --- 1 = 用于 FET 温度保护 OPT[0]： --- 0 = 未用于电芯温度保护 --- 1 = 用于电芯温度保护
ddsg_pin_fxn	[1: 0]	0x0	RW	配置此引脚用于哪项功能。 0 =RSV 1 = 通用输出 2 = DDSG 3 = ADC 输入或热敏电阻

2.3.5 NTC 测量引脚的配置

针对 TS1,TS2 和 TS3 作为温度测量的设置，合并到了 2 个寄存器中。例如如果需要设置 TS1 和 TS3 作为 NTC 测量，内部上拉电阻并且使用 NTC 作为电芯保护，设置 TS1_CONFIG(0X8209) 为 0x50，设置 TS23_CONFIG(0X820A) 为 0x50。

关于 NTC 采样设置，除了相应的 GPIO 设置上，需要多功能引脚的 pin_fxn 设置为 3，OPT[2]设置为 1 选择上拉电阻之外。还需要总体设置 PAD_CFG(0x8201)的 BIT[5:4]，设置选择上拉电阻为无，18k 还是 180k。同时需要设置上拉电阻温度系数参数：COEFF_A1_18K(0x8448), COEFF_A2_18K(0x844A), COEFF_A3_18K(0x844C), COEFF_A4_18K(0x844E)。以 18k 上拉电阻，选择典型的参考配置值：Reg0x8448=0xEC67, Reg0x844A=0x4262, Reg0x844C=0xF6D7, Reg0x 844E=0x04C4.

TS1 的配置：

Register: TS1_CONFIG | Address: 0x00008209				
Field	**Bits**	**Default**	**Type**	**Field Description**
ts1_opt	[7: 4]	0x0	RW	OPT[3]是否为 ADC 输入： 0 = 否 1 = 是 OPT[2]是否为外部 Tsensor 输入： ---0 = 否 ---1 = 是 OPT[1]： --- 0 = 未用于 FET 温度保护 --- 1 = 用于 FET 温度保护 OPT[0]： --- 0 = 未用于电芯温度保护 --- 1 = 用于电芯温度保护

TS23 的配置：

Register: TS2_3_CONFIG | Address: 0x0000820a				
Field	**Bits**	**Default**	**Type**	**Field Description**
ts3_opt	[7: 4]	0x0	RW	OPT[3]是否为 ADC 输入： 0 = 否 1 = 是 OPT[2]是否为外部 Tsensor 输入： ---0 = 否 ---1 = 是 OPT[1]： --- 0 = 未用于 FET 温度保护 --- 1 = 用于 FET 温度保护 OPT[0]： --- 0 = 未用于电芯温度保护 --- 1 = 用于电芯温度保护
ts2_opt	[3: 0]	0x0	RW	OPT[3]是否为 ADC 输入： 0 = 否 1 = 是 OPT[2]是否为外部 Tsensor 输入： ---0 = 否 ---1 = 是 OPT[1]： --- 0 = 未用于 FET 温度保护 --- 1 = 用于 FET 温度保护 OPT[0]： --- 0 = 未用于电芯温度保护 --- 1 = 用于电芯温度保护

PAD_CFG 的配置：

Register: PAD_CFG | Address: 0x00008210				
Field	**Bits**	**Default**	**Type**	**Field Description**

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

RSV	[7: 7]	0x1	RW	RSV
da_fuse_oe	[6: 6]	0x0	RW	0：FUSE 管脚做为输入管脚 1：FUSE 管脚做为输出管脚
ts_res_sel	[5: 4]	0x0	RW	0 = 无上拉 1 = Tsensor 输入，18k 上拉 2 = Tsensor 输入，180k 上拉 3 = 未使用
RSV	[3: 0]	0x0	RW	RSV

2.3.6 保护阈值与延迟的 STEP 的差异

芯片提升了 CELL 硬件保护的阈值，将保护 step 提升到 20mv，与 BQ76952 存在差异，所以计算 cov_th，cuv_th 的阈值 STEP 为 20mV，并且设置值存在一个 offset。举例，如果设置 cov_th 为 4.2V，计算值为 4200/20 - 1=209=0xD1，设置保护阈值为 0xD1。保护阈值迟滞电压也是 20mV 为一个 step。

由于内部时钟的差异，cov_delay,cuv_delay,occ_delay,ocd1_delay,ocd2_delay 延迟时间 step 为 3.9ms，与 BQ76952 的 3.3ms 不同。并且上述值存在 2 个 offset。比如设置 cov_delay 为 0，就以为发生 cov 事件 3.9*2=7.8ms 后，就会触发保护。

2.3.7 CHG_PUMP_CONTROL 配置

CHG_PUMP_CONTROL 中驱动电平的设置 TI 的 BQ76952 相反，如果配置的电平不合适， MOS 的驱动电压可能不够。Bit[1]: lven 配置为 1 时，chug pmup 为 11V 的输出，配置为 0 时为 5.5V 的输出。

CHG_PUMP_CONTROL | Address: 0x00008825				
Field	**Bits**	**Default**	**Type**	**Field Description**
sleep_cpen	[3:3]	0x1	RW	0: Charge pumps for FET drivers are disabled in sleep mode 1: Charge pumps for FET drivers are enabled in sleep mode
sfmode_sleep	[2:2]	0x0	RW	0: Source-follower mode is not enabled on the DSG FET driver 1: Source-follower mode is enabled on the DSG FET driver while in SLEEP mode
lven	[1:1]	0x0	RW	This bit selects the charge pump overdrive level. 0: Charge pump low overdrive level (5.5 V) is selected 1: Charge pump high overdrive level (11 V) is selected
normal_cpen	[0:0]	0x1	RW	0: Charge pumps for FET drivers are disabled in normal mode 1: Charge pumps for FET drivers are enabled in normal mode

2.3.8 电流寄存器差异

- DA_CONFIGURATION Register 与 BQ75952 不一致，BM2016 取消了 DA_CONFIGURATION 中的 userA 的配置寄存器，userA 的大小由写入的 CC GAIN 寄存器决定，这样 userA 的范围更加灵活，用户可以根据采样电阻和最大电流计算出最合适的 userA。

DA_CONFIGURATION Register | Address: 0x00008807				
Field	**Bits**	**Default**	**Type**	**Field Description**
da_configuration	[7:0]	0x5	RW	[1:0] : reserved [2] : userV define To ensure the Top-of-Stack, PACK, and LD pin voltages fit in a signed 16-bit integer type, their units are configurable. This configurable unit is called user-volts and can be

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

				set to either centivolts or millivolts. For applications which will not exceed 32 Volts, millivolts may be used. Other applications should use centivolts to avoid saturating at 32767 mV. 0 = Millivolt (1 mV) units are selected for user-volts 1 = Centivolt (8 mV) units are selected for user-volts [3] : TINT_EN enables the internal temperature source to be used as the “CellTemperature” for protections and logic that use minimum, maximum, or average temperature. 0:Internal temperature is not used for "Cell Temperature" 1:Internal temperature is used for "Cell Temperature" [4] : TINT_FETT enables the internal temperature source to be used as the "FET Temperature". TINT_EN must also be set for this to apply. When TINT_EN = 0, this bit is ignored. When configured for FET temperature, it is not factored into minimum, maximum, and average cell temperature calculations. 0:Internal temperature is not used for "FET Temperature" 1:Internal temperature is used for "FET Temperature" if TINT_EN is also set Others : RESERVED.

- 与 BQ76952 类似，BM2016 也提供了 CC1/2/3 Current 和 32 位的 ADC count 测量值，其中 CC2 为实时电流，其中 CC1 Current 为固定 100 个样本 CC2 Current 的平均值，CC3 Counts 为可编程的 CC2 电流的平均值的 24 bit 平均值(低 8bit 为小数），CC3 Current 为 CC3 Count 的高 16bit。CC2 Counts 为 SRP-SPN 的电压采样值，单位为 28.896nV, CC2 avg count 为可编程数量的 CC2 COUNTS 的平均值。此次做个说明，BM2016 内部电流 ADC 是 16bit，ADC 转换后还有 CIC 抽取滤波器，滤波后的数据为 24bit，实际上电流 ADC 精度已经是 24bit，噪声被多次滤波处理，小电压下噪声电压在 5uV 数量级。用户如果没有很高平均次数的需求，不建议使用 CC3 进行计算。

- 需要注意的是以上所有的电流返回值都带有符号位，当以上寄存器的返回值最高符号位为 1 时，代表此时电流为负值，需要先转换为补码获取真实的电流或者 ADC 采样值，寄存器定义具体如下所示。

Reg 名称	Address	Reg 描述	Reg 长度	符号位
DAStatus5_11 Register	0x0116	CC1 Current，100 个 CC2 Current 的平均值	16bit	第 16bit
CC2 Current Register	0x3A	CC2 Current, 瞬时电流的测量	16bit	第 16bit
DAStatus5_10 Register	0x0114	CC3 Current, CC3 Counts 的高 16bit 结果，单位与 CC2 Current 一致	16bit	第 16bit
CC3_SAMP Register	0x8461	决定 CC2 Current 样本数量以计算 CC3 Counts，默认为 80	8bit	/
DAStatus5_12 Register	0x0118	CC2_Counts, SRP-SPN 瞬间 ADC measurement value，单位是 28.896nV	32bit	第 32bit
Read Cal1_0 Register	0x0260	CC2_AVG_Counts，可编程数量的 CC2_Counts 24bit 平均值	24bit	第 24bit
CC_OFFSET_SAMP	0x843A	决定 CC2 counts 样本数量以计算 CC2_AVG_Counts，默认为 64	16bit	/

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

DAStatus5_13 Register	0x011C	CC33_Counts,可编程数量的 CC2 Current 的 24bit 平均值，高 16bit 为整数，低 8bit 为小数。	32bit	第 32bit

2.4 软件配置差异

2.4.1 通信差异

- BM2016 支持两种串行通信接口，分别为 I2C、SPI 接口。I2C 接口支持最高 400kHz 时钟速率（可选 CRC 校验）；SPI 接口为从接口，支持高达 2MHz 的时钟速率（可选 CRC 校验）。BM2016 在 I2C 模式下支持两个设备地址默认地址为 0x20，其设备地址由 SPI_MOSI 引脚决定，引 SPI_MOSI 引脚 floating 或者拉低设备地址为 0x20，引脚拉高设备地址为 0x22。
- BM2016 第一次上电，通信时钟时不工作，需要用通信时钟（SCL 或者 SPI_CS）唤醒，所以用户上电第一次发送通信 AFE 是 nack 的，第二次 AFE 才会 ack 主机的命令。
- BM2016 通信的帧格式、CHECK SUM 计算以及 CRC 计算方式和 BQ76952 有部分差异，具体的差异和示例请查阅 *GD30BM2016 软件用户指南*文档，该文档详细描述了直接子命令、不带数据子命令、带数据子命令、寄存器读写的 I2C/SPI 帧格式命令，CheckSum 计算、CRC 计算和相关示例。CRC 数据清零复位的逻辑与 TI 不同，BM2016 只有在计算 CRC 数据后才会复位 CRC，开始下一次新的计算。例如主机之前在与其他设备通信，之后要切换与 AFE 通信时需要先任意发送一次读或者写命令来复位 CRC。
- 对于直接子命令读，BM2016 不支持一次通信连续读 16 节电芯电压，如果用户要获取电压、温度等数据，需要按照下面的帧格式读取（以 I02 为例），每次只能返回一个命令对应的寄存器值。
- 在执行带数据的子命令或者寄存器读命令时必须等待足够的时间来让器件完成操作，再向 0x61 写完要读取的长度后，等待至少 200us，再发送 0x40 读指令获取数据

S	**SLV_ADDR**	**W**	**A**	**Dir CMD**	**A**	**RS**	**SLV_ADDR**	**W**	**A**	**BYTE0**	**A**	**BYTE1**	**A**	**CRC8**

2.4.2 电压串数

BQ76952 上电默认为 16 节电芯工作，BM2016 默认只有 CELL1 工作，用户如果上电为 16 节，需要将 VCELL_MODE 0x8808 寄存器配置为 0xFEFF，bit 8 必须配置为 0。如果串数低于 16 串，需要将未使用的 cell 短接，并且配置 VCELL_MODE 对应为 0。另外 CELL1 和 CELL16 必须使用，不可以将 CELL16 短接到 CELL15 上，同理 CELL1 也不能短接到 CELL2 上，否则最低串、最高串的均衡会被影响。

2.4.3 电流增益设置和读取

关于电流增益的设置，BQ76952 需要配置 CC GAIN 和 user 属性，BM2016 根据最大电流和采样电阻值，提供表格计算 userA，并根据 userA 生成 CC GAIN。

CC_GAIN Register | Address: 0x842C				
Field	**Bits**	**Default**	**Type**	**Field Description**
cc_gain	[19:0]	0xbb8	RW	This parameter sets the gain factor used to convert coulomb counter raw count measurements to current. The value is given by Calibration:Current:CC Gain = 30 / (Rsense in Ohm) / UserA. User calculates userA based on the actual connected resistance, referring to the porting guide.

电流增益 CC gain 的计算与采样电阻的大小、userA 的单位均有关系，GD 提供表格计算 userA, userA 用户可以自定义，建议选择 1、2、4、6、8、16、32、64、128 档位，用户可以选择合适的 userA 避免电流采样计算结果溢出，也支持任意灵活的 userA 可以提高电流采样精度。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

下面介绍了 userA 的计算原理：假设 userA 为 1，CC_GAIN = 30/ Rsense_ohm，当采样电阻为 0.1mohm 时计算结果为 30/0.0001=300000，若配置 CC_GAIN 为 300000 时，在计算 16bit 的 CC2 Current 过程中很容易溢出，因此需要计算出合适的 UserA 避免计算溢出问题，userA 其计算过程如下：

- 计算电流采样 1mV 对应的 AVG Count 值，其中 6851336 为 SNP-SNR 压降 200mV 时，电流采样 Adc 的 24 bit 采样值：

$$\frac{6851336}{200} \approx 34256.7$$

- 计算最大电流对应的 AVG $Count_{max}$，假设用户采用 0.1mohm 的电阻，最大电流 600A：

$$\mathrm{RoundUp}(0.1 * 600 * 34256.7) \approx 2055360$$

- 先选择 userA 为 1，计算出 userA 需要放大的倍数 K： AVG $Count_{max}$*30/ Rsense_ohm/(2^35)，其中 2^35 为 AVG count * CC gain 的最大值，若 CC gain 和 Avg CC2 Counts 乘积超过了 2^35，会导致 16 位的 CC2 Current 结果溢出。

$$\mathrm{K} = \frac{\mathrm{AVG\ Countmax} * 30/\ \mathrm{Rsense_ohm}}{(2\text{^}35)} = \frac{2055360 * 30/0.0001}{2\text{^}25} \approx 17.9$$

- UserA 最少要选择大于 17.9 才不会出现溢出的情况(向上取整)，表格中 userA 需要配置为 2 的 n 次方，所以建议 userA 选择 32，若选择 64、128 虽然不会溢出，但是会降低 CC2 Current 的精度。需要注意的是假设计算出的 userA 结果为 15.9，这时候建议 userA 不要选择 16，太临界的值在有扰动时容易有溢出，选择 20-32 是一个安全的值。

计算出 userA 后，可以去计算合适的 CC_GAIN 值，按照计算公式 CC_GAIN = 30/ RSENSE_mohm /userA，获得 CC_GAIN 后并配置，CC2 Current 的单位为 userA*1mA。例如如果采样电阻的值 1mohm，设置 userA 为 8mA，CC2_GAIN = 30/0.001/8 = 3750。通过直接命令读取到 CC2_Current 直接乘以 8mA，得到就是实际的电流值，注意 CC2_Current 的值是有符号数据，大于 32768 代表是负数，对应是放电电流，小于 32768 是整数，代表充电电流。

2.4.4 寄存器和命令解密

- BQ76952 系统设计了寄存器解锁的 SEAL KEY，但是即使不开启 SEAL_KEY，仍然可以做寄存器的更新。BM2016 为了配置安全要求，必须要输入密码进入 full access 模式后，再进入 set config 模式才可以配置寄存器，退出 config 模式后寄存器才会生效。
- 用户若使用 MCU 发送命令控制 MOS，建议要在 full access 模式下发送，例如 PDSG_TEST、PCHG_TEST、CHG_TEST、DSG_TEST 在 SEAL 模式下被 AFE 忽略，在 full access 模式下才可以生效。

3 BM2016 常用功能注意事项

3.1 BM2016 上电时序

GD30BM2016 的 RST_SHUT 引脚控制芯片进入 shutdown 模式，TS2 引脚下降沿将芯片从 shutdown 模式唤醒至 normal 模式，AFE 在上电后，用户需要发送通信唤醒 AFE 通信模块的时钟，在 sleep disable 模式下配置 AFE，具体操作参考下面时序图。

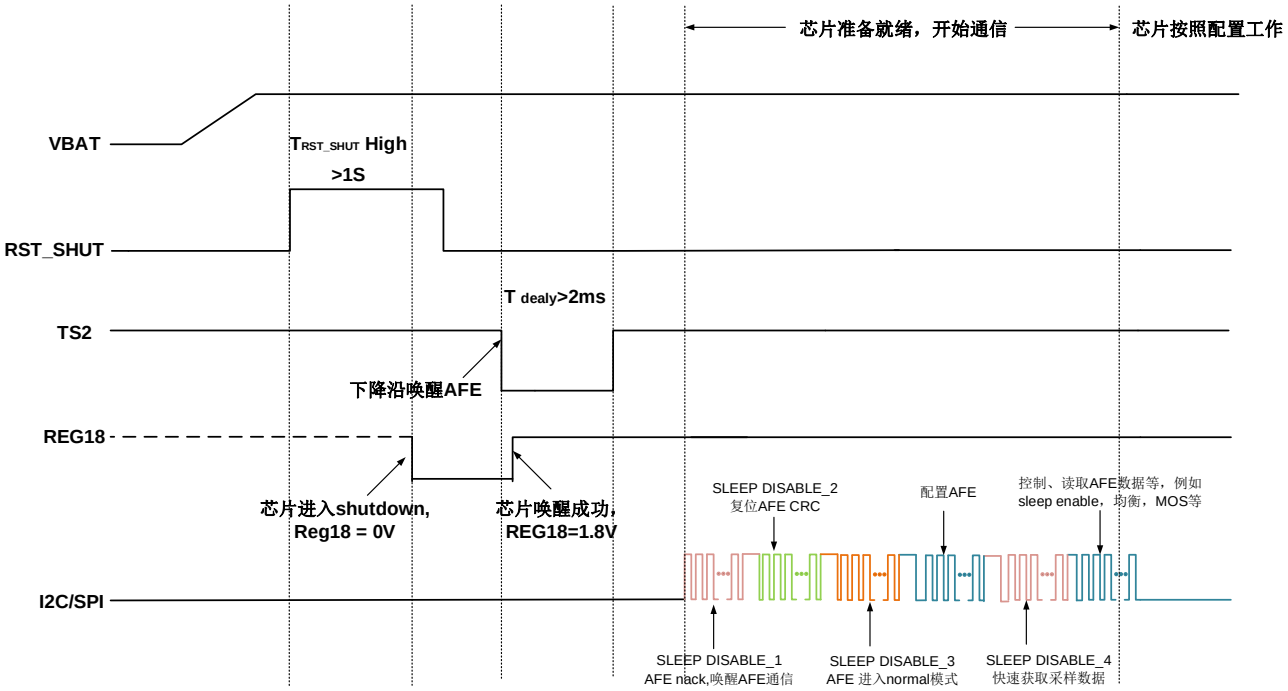
3.1.1 硬件复位芯片

MCU 先配置 RST_SHUT PIN 为低， WAKE UP PIN 为高电平（TS2 内部若上拉为高电平，如果 MCU IO 输出低电平则会有漏电）。也有许多用户在 WAKE UP PIN 上接入三极管，到 TS2 的信号与 WAKE PIN 信号相反，这种硬件下 WAKE UP PIN 需要先配置为低电平。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

AFE 上电后 MCU 进入 AFE 初始化，先将 RST_SHUT 引脚拉高大于 1s，令 AFE 进入 shutdown 模式，再将 RST_SHUT 引脚拉低且长期保持低，芯片才可以后续正常工作，等待至少约 10ms 后向 TS2 引脚灌入下降沿以唤醒 AFE，保持 TS2 引脚低 2ms 后再拉高 TS2 引脚，AFE 已经被成功复位并唤醒。

接着 MCU 发送任意一个通信命令，建议发送 sleep disable 唤醒 AFE 的通信时钟，如果是 I02 芯片或者 I04 芯片建议再次发送一次 sleep disable 命令复位 CRC，如果不带 CRC 可以取消这一步，完成后通信已经准备就绪。再次发送 sleep disable 命令，芯片进入 normal 模式，在该模式下配置 AFE 寄存器。用户也可以通过直接子命令读 Battery Status Register: bit [2]确认 sleep_en 被关掉，bit[15]确认芯片退出 sleep 模式。



AFE 上电硬件复位时序图

Sleep Disable Register | Address: 0x0354				
Field	**Bits**	**Default**	**Type**	**Field Description**
sleep_disable	[7:0]	0x0	WO	sub cmd only

Battery Status Register | Address:0x12				
Field	**Bits**	**Default**	**Type**	**Field Description**
sleep	[15:15]	0x0	RO	This bit indicates whether or not the device is presently in SLEEP mode. 0 = Device is not in SLEEP mode. 1 = Device is in SLEEP mode.
sec	[9: 8]	0x0	RO	These bits indicate the present security state of the device. When in SEALED mode, device configuration may not be read or written and some commands are restricted. When in UNSEALED mode, device configuration may normally be read and may be written while in CONFIG_UPDATE mode. When in FULLACCESS mode, unrestricted read and write access is allowed and all commands are accepted. Even in FULLACCESS mode, changes to device

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Battery Status Register | Address:0x12				
Field	**Bits**	**Default**	**Type**	**Field Description**
				configuration should only be changed while also in CONFIG_UPDATE mode. 0 = Device has not initialized yet. 1 = Device is in FULLACCESS mode. 2 = Device is in UNSEALED mode. 3 = Device is in SEALED mode
sleep_en	[2:2]	0x0	RO	This bit indicates whether or not SLEEP mode is allowed based on configuration and commands. The Settings:Configuration:Power Config[SLEEP] bit sets the default state of this bit. The host may send commands to enable or disable SLEEP mode based on system requirements. When this bit is set, the device may transition to SLEEP mode when other SLEEP criteria are met. 0 = SLEEP mode is disabled by the host. 1 = SLEEP mode is allowed when other SLEEP conditions are met.
cfgupdate	[0:0]	0x0	RO	This bit indicates whether or not the device is in CONFIG_UPDATE mode. It will be set after the SET_CFGUPDATE command is received and fully processed. Configuration settings may be changed only while this bit is set. 0 = Device is not in CONFIG_UPDATE mode. 1 = Device is in CONFIG_UPDATE mode.

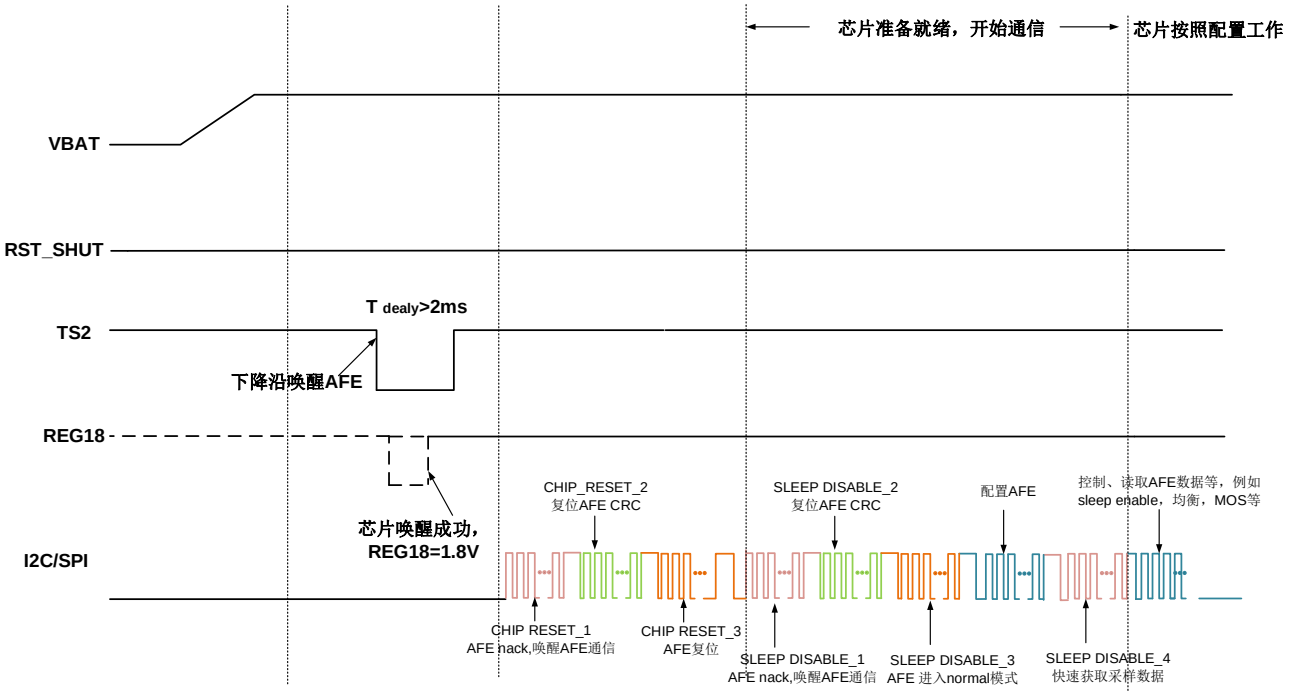
3.1.2 软件复位芯片

优先推荐使用硬件复位芯片模式进行初始化操作，但有些用户对于上电初始化时间有更快的要求，无法接受将 RST_SHUT 拉高 1S，此时可以使用命令复位芯片。具体操作如下：

1. MCU 先配置 RST_SHUT PIN 为低， WAKE UP PIN 为高电平如果 WAKE UP IO 到 TS2 未取反。
2. AFE 上电后 MCU 进入 AFE 初始化，继续保持 RST_SHUT 为低，先将 TS2 引脚灌入下降沿以唤醒 AFE，保持 TS2 引脚低 2ms 后再拉高 TS2 引脚，AFE 已经被成功复位并唤醒，向 AFE 连发三次 CHIP_RESET 命令（确保芯片复位，如果不是 I02 或者 I04，发送 2 次 CHIP_RESET 命令即可）。
3. 发送 sleep disable 唤醒 AFE 的通信时钟，如果是 I02 芯片或者 I04 芯片建议再次发送一次 sleep disable 命令复位 CRC，如果不带 CRC 可以取消这一步，完成后通信已经准备就绪，再次发送 sleep disable 命令。芯片进入 normal 模式，在该模式下配置 AFE 寄存器

Chip Reset Register | Address : 0x03CC				
Field	**Bits**	**Default**	**Type**	**Field Description**
chip_reset	[7:0]	0x0	WO	sub cmd only, this command will reset the chip.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.



AFE 上电软件复位时序图

3.2 BM2016 配置模式

用户要配置 AFE，在完成 3.1 章节上电后，需要先解锁进入 Full Access 模式，在此模式下发送 set config 命令，此时 AFE 才允许正常的写寄存器。

1. 用带数据子命令格式先向 0x0280 寄存器写入 0x5A5A5A5A，输入第一个密码进入 unseal 模式；
2. 用带数据子命令格式向 0x0280 寄存器写入 0XA5A5A5A5，输入第二个密码，芯片进入 full_access 模式，用户也可以读 Battery Status Register :sec bit[9:8]确认；
3. 发送 SET Cfgupdate 命令进入配置模式，用户也可以读 Battery Status Register :cfgupdate bit[0]确认命令生效；
4. 按照用户需求配置对应的寄存器；
5. 配置结束后发送 Exit Cfgupdate 退出配置模式，寄存器生效。

Security Access Register | Address: 0x280				
Field	**Bits**	**Default**	**Type**	**Field Description**
access_key_step2	[31:16]	0xff	RWAC	Access Key Step 2
access_key_step1	[15: 0]	0xff	RWAC	Access Key Step 1

SET Cfgupdate Register | Address: 0x0338				
Field	**Bits**	**Default**	**Type**	**Field Description**
set_cfgupdate	[7:0]	0x0	WO	sub cmd only

Exit Cfgupdate Register | Address: 0x033C				
Field	**Bits**	**Default**	**Type**	**Field Description**
exit_cfgupdate	[7:0]	0x0	WO	sub cmd only

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

3.3 电压配置和寄存器

确定芯片外围连接的电芯串数后，首先配置 VCELL_MODE Register(0x8808)使能电芯采样和保护，配置生效后通过电芯电压子命令获取电芯电压采样结果，用户也可以通过带数据的子命令获取电芯电压 ADC 采样的原始码值。如果用户采用不同的 VCX 输入电阻，通过修改 CELL GAIN 提高电芯电压精度。此外 GD30BM2016 也提供了 Stack、PACK、LD 电压的采样值，也加工了最小电芯电压、最大电芯电压和所有电芯电压的和。

VCELL_MODE | Address: 0x00008808
Cell[x] Mode Enable Bit。具体描述查看寄存器文档

Cell1 Voltage Register - Cell16 Voltage Register： 16-bit voltage on cell 1, the unit is mV							
Cell Volt 1	Cell Volt 2	Cell Volt 3	Cell Volt 4	Cell Volt 5	Cell Volt 6	Cell Volt 7	Cell Volt 8
0x14	0x16	0x18	0x1A	0x1C	0x1E	0x20	0x20
Cell Volt 9	Cell Volt 10	Cell Volt 11	Cell Volt 12	Cell Volt 13	Cell Volt 14	Cell Volt 15	Cell Volt 16
0x24	0x28	0x2A	0X2C	0X2E	0X30	0X32	0X3C

Cell1 Volt_Counts- Cell16 Volt_Counts： 32-bit ADC counts for cell voltage measurement, the unit is 0.722uV.							
cell1 counts	Cell2 counts	Cell3 counts	Cell4 counts	Cell5 counts	Cel6 counts	Cell7 counts	Cell8 counts
0x0080	0x0088	0x0090	0x0098	0x00A0	0x00A8	0x00B0	0x00B8
Cell9 counts	Cell10 counts	Cell11 counts	Cell12 counts	Cell13 counts	Cell14 counts	Cell15 counts	Cell16 counts
0x00C0	0x00D0	0x00D8	0X00E0	0X00E8	0X00F0	0X00F8	0X0154

CELL1_GAIN Register - CELL16_GAIN Register： Cell voltage calculations use a linear gain set by this value. Gain is factory-calibrated but can be modified by adjusting this value.							
Cell1_GAIN	Cell2_GAIN	Cell3_GAIN	Cell4_GAIN	Cell5_GAIN	Cell6_GAIN	Cell7_GAIN	Cell8_GAIN
0x8400	0x8402	0x8404	0x8406	0x8408	0x840A	0x840E	0840E
Cell9_GAIN	Cell10_GAIN	Cell11_GAIN	Cell12_GAIN	Cell13_GAIN	Cell14_GAIN	Cell15_GAIN	Cell16_GAIN
0x8410	0x8414	0x8416	0X8418	0X841A	0X841C	0X841E	0X8420

Stack Voltage Register | Address : 0x34				
Field	**Bits**	**Default**	**Type**	**Field Description**
stack_volt	[15:0]	0x0	RO	16-bit voltage on top of stack, the unit is userV, default is 8mV

Pack Pin Voltage Register | Address : 0x36				
Field	**Bits**	**Default**	**Type**	**Field Description**
pack_pin_volt	[15:0]	0x0	RO	16-bit voltage on PACK pin, the unit is userV, default is 8mV

LD Pin Voltage Voltage | Address : 0x38				
Field	**Bits**	**Default**	**Type**	**Field Description**
ld_pin_volt	[15:0]	0x0	RO	16-bit voltage on LD pin, the unit is userV, default is 8mV

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Read Cal1_2 Register | Address : 0x0268				
Field	**Bits**	**Default**	**Type**	**Field Description**
pack_pin_adc_cnt	[15:0]	0x0	RO	16-bit ADC counts from previous PACK pin voltage measurement.

Read Cal1_3 Register | Address : 0x026A				
Field	**Bits**	**Default**	**Type**	**Field Description**
top_stack_adc_cnt	[15:0]	0x0	RO	16-bit ADC counts from previous Top of Stack voltage measurement.

Read Cal1_4 Register | Address : 0x026C				
Field	**Bits**	**Default**	**Type**	**Field Description**
ld_pin_adc_cnt	[15:0]	0x0	RO	16-bit ADC counts from previous Top of LD pin measurement

DAStatus5_2 Register | Address : 0x0104				
Field	**Bits**	**Default**	**Type**	**Field Description**
max_cell_volt	[15:0]	0x0	RO	Maximum Cell Voltage, the unit is mV.

DAStatus5_3 Register | Address : 0x0106				
Field	**Bits**	**Default**	**Type**	**Field Description**
min_cell_volt	[15:0]	0x0	RO	Minimum Cell Voltage, the unit is mV.

DAStatus5_4 Register | Address :0x0108				
Field	**Bits**	**Default**	**Type**	**Field Description**
battery_volt_sum	[15:0]	0x0	RO	Sum of cell voltages (including interconnects). This can be compared to the Stack Voltage() for diagnostic purposes. Note, however, that these measurements are taken at different times which may cause variation , the unit is userV, default is 8mV.

3.4 电流配置和寄存器

用户配置和读取电流按照一下顺序：

1. 根据之前的描述，用户需要先根据 GD 提供的表格计算出 userA，再计算合适的 CC gain 写入寄存器中；
2. 如果精度满足要求，则可以忽略第二步和第三步，如果发现有较大的电流偏置，用户需要短接采样电阻获取当前真实的偏置，建议读取 CC2_AVG_COUNT 寄存器获取偏置值，再写入 CC2 OFFSET 寄存器校准偏置；
3. 如果用户电阻有偏差，需要读两个不同电流点的 CC2_AVG_COUN 差值来校准 CC gain；
4. 配置完电流寄存器后读电流返回值寄存器获取电流。
5. BM2016 支持库仑计功能，用户也可以通过带数据的子命令访问库仑计结果，如果用户使用该功能，需要定时的发送库仑计计数清零的命令 Reset PassQ ，避免长时间累计导致溢出。

CC_GAIN Register | Address: 0x842C				
Field	**Bits**	**Default**	**Type**	**Field Description**
cc_gain	[19:0]	0xbb8	RW	This parameter sets the gain factor used to convert coulomb counter raw count measurements to current. The value is given by

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

				Calibration:Current:CC Gain = 30 / (Rsense in Ohm) / UserA.User calculates userA based on the actual connected resistance, referring to the porting guide.

CC2_OFFSET | Address: 0x8432				
Field	**Bits**	**Default**	**Type**	**Field Description**
cc2_offset	[15:0]	0x0	RW	This parameter sets the offset factor used to convert coulomb counter raw count measurements to passed charge in normal mode. The unit is 0.05mA.

Reg 名称	Address	Reg 描述	Reg 长度	符号位
DAStatus5_11 Register	0x0116	CC1 Current，100 个 CC2 Current 的平均值	16bit	第 16bit
CC2 Current Register	0x3A	CC2 Current, 瞬时电流的测量	16bit	第 16bit
DAStatus5_10 Register	0x0114	CC3 Current, CC3 Counts 的高 16bit 结果，单位与 CC2 Current 一致	16bit	第 16bit
CC3_SAMP Register	0x8461	决定 CC2 Current 样本数量以计算 CC3 Counts，默认为 80	8bit	/
DAStatus5_12 Register	0x0118	CC2_Counts, SRP-SPN 瞬间 ADC measurement value, 单位是 28.896nV	32bit	第 32bit
Read Cal1_0 Register	0x0260	CC2_AVG_Counts，可编程数量的 CC2_Counts 24bit 平均值	24bit	第 24bit
CC_OFFSET_SAMP	0x843A	决定 CC2 counts 样本数量以计算 CC2_AVG_Counts，默认为 64	16bit	/
DAStatus5_13 Register	0x011C	CC33_Counts,可编程数量的 CC2 Current 的 24bit 平均值，高 16bit 为整数，低 8bit 为小数。	32bit	第 32bit

DAStatus6_0 Register Address: 0x120

Field	Bits	Default	Type	Field Description
accum_charge	[31:0]	0x0	RO	Reports the integer portion of accumulated passed charge in userAmp-hours.

DAStatus6_1 Register Address: 0x124

Field	Bits	Default	Type	Field Description
accum_charge_fraction	[31:0]	0x0	RO	Reports the fractional portion of accumulated passed charge. This is initialized to 0.5 userAh to facilitate appropriate rounding of the integer portion.

DAStatus6_2 Register Address : 0x128

Field	Bits	Default	Type	Field Description
accum_time	[31:0]	0x0	RO	Reports the number of seconds over which passed charge has been integrated. The unit is S.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Reset PassQ Register Address: 0x30

Field	Bits	Default	Type	Field Description
reset_passq	[7:0]	0x0	WO	sub cmd only

3.5 温度配置和读取

温度配置用户可以参考下面步骤进行配置：

- 根据外围电路热敏电阻的连接情况，按照寄存器定义配置对应的引脚为热敏电阻输入，同时配置是否需要打开电芯温度保护或者 FET 温度保护，例如选择 TS1 和 SPI_MOSI 引脚作为热敏电阻输入，需要 TS0_1 Config 配置为 0x50，多功能引脚 SPI_MOSI 配置为 0x13，多功能引脚的定义查看寄存器的 Pad 章节。
- 引脚配置结束后，需要配置是否使用芯片内部上拉和上拉电阻的阻值，具体查看 PAD CFG Register(0x8210)寄存器定义,这里配置的是所有热敏电阻的上拉源和上拉电阻，如果用户使用内部上拉 ts_res_sel[5:4]需要配置为 1 或者 2，如果用户想使用外部上拉，ts_res_sel[5:4]要配置为 0。
- 最后根据热敏电阻型号和 GD 提供的表格，计算需要写入的温度热敏电阻系数，将结果写入温度系数寄存器 COEFF_A1_18K Register - COEFF_A4_18K Register，如果使用的是 180K 上拉电阻，写入 COEFF_A1_180K Register - COEFF_A4_180K Register。
- 配置结束后可以通过直接子命令访问读取温度，温度单位为 0.1℃，如果用户需要获取温度引脚 ADC 采样结果，也可以通过带数据的子命令获取。

温度配置需要注意以下几点：

- TS1 必须默认用作温度热敏电阻，并且作为电芯温度保护，因此必须 TS0_1 Config Register 为 0x50，如果配置为 0x00 或者 0x40 会误报 UTC 告警，并且导致 UTD、OTD 温度告警错误无法恢复。
- 如果 TS2 配用作热敏电阻，按照默认值 TS2 引脚同时也为硬件唤醒，由于 TS2 对地有热敏电阻，芯片无法进入 shutdown 模式这里处理和 TI 一致，如果用户要使用 TS2 引脚作为热敏电阻，还要进入 shutdown 模式，可以将 Power Config Register(0x8000) bit[12]：shut_ts2:烧录为 1 即 LD 唤醒，这样可以同时满足 TS2 用作热敏电阻且 shutdown 功能正常。
- DCHG 引脚和 DDSG 引脚在作为温度热敏电阻输入时，需要配置 0x8E2A 寄存器对应的 bit 断开 MOS 状态监测影子寄存器功能，并且选择为电芯温度保护。当 0x8e2a[0]=0 时，DCHG 作为 CHG 的影子信号，当 0x8e2a[1]=1 时，DDSG 作为 DSG 的影子信号，如果作为温度输入需要配置为 1.

备注：若要获取温度 PIN 配置寄存器详细细节，请按照寄存器名称在 *Register Maps* 寄存器文章中查找，所有温度读取数据寄存器如下所示。

PAD CFG Registe | Address: 0x8210				
Field	**Bits**	**Default**	**Type**	**Field Description**
ts_res_sel	[5: 4]	0x0	RW	0 = No pull-up 1 = Tsensor input, 18k pull-up 2 = Tsensor input, 180k pull-up 3 = Not used

INT Temperature Register：Address: 0x68

Field	Bits	Default	Type	Field Description
int_temp	[15:0]	0x0	RO	This is the most recent measured internal die temperature, the unit is 0.1℃.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

CFEOFF Temperature Register Address: 0x6A

Field	Bits	Default	Type	Field Description
cfetoff_temp	[15:0]	0x0	RO	When the CFETOFF pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the CFETOFF pin in millivolts. the unit is 0.1℃.

DFEOFF Temperature Register Address: 0x6C

Field	Bits	Default	Type	Field Description
dfetoff_temp	[15:0]	0x0	RO	When the DFETOFF pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the DFETOFF pin in millivolts. the unit is 0.1℃.

Alert Temperature Register Address: 0x6E

Field	Bits	Default	Type	Field Description
alert_temp	[15:0]	0x0	RO	When the ALERT pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the ALERT pin in millivolts. the unit is 0.1℃.

TS1 Temperature Register Address: 0x70

Field	Bits	Default	Type	Field Description
ts1_temp	[15:0]	0x0	RO	When the TS1 pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the TS1 pin in millivolts. the unit is 0.1℃.

TS2 Temperature Register Address: 0x72

Field	Bits	Default	Type	Field Description
ts2_temp	[15:0]	0x0	RO	When the TS2 pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the TS2 pin in millivolts. the unit is 0.1℃.

TS3 Temperature Register Address: 0x74

Field	Bits	Default	Type	Field Description
ts3_temp	[15:0]	0x0	RO	When the TS3 pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the TS3 pin in millivolts. the unit is 0.1℃.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

SPI MOSI Temperature Register Address: 0x76

Field	Bits	Default	Type	Field Description
spi_mosi_temp	[15:0]	0x0	RO	When the SPI_MOSI pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the SPI_MOSI pin in millivolts. the unit is 0.1℃.

DCHG Temperature Register Address : 0x78

Field	Bits	Default	Type	Field Description
dchg_temp	[15:0]	0x0	RO	When the DCHG pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the DCHG pin in millivolts. the unit is 0.1℃.

DDHG Temperature Register Address: 0x7A

Field	Bits	Default	Type	Field Description
ddsg_temp	[15:0]	0x0	RO	When the DDSG pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the DDSG pin in millivolts. the unit is 0.1℃.

TS0 Temperature Register :Address Offset: 0x7C

Field	Bits	Default	Type	Field Description
ts0_temp	[15:0]	0x0	RO	When the TS0 pin is configured as a thermistor input, this reports its most recent temperature measurement. When configured as ADCIN, this instead reports the measured voltage at the TS0 pin in millivolts. the unit is 0.1℃.

DAStatus6_3 Register Address : 0x12C

Field	Bits	Default	Type	Field Description
cfetoff_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the CFETOFF pin. The unit is 0.358uV.

DAStatus6_4 Register Address : 0x130

Field	Bits	Default	Type	Field Description
dfetoff_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the DFETOFF pin. The unit is 0.358uV.

DAStatus6_5 Register Address : 0x134

Field	Bits	Default	Type	Field Description
alert_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the ALERT pin. The unit is 0.358uV.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

DAStatus6_6 Register Address : 0x138

Field	Bits	Default	Type	Field Description
ts1_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the TS1 pin. When configured for thermistor measurement, the unit is 0.358uV.

DAStatus6_7 Register Address t: 0x13C

Field	Bits	Default	Type	Field Description
ts2_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the TS2 pin. The unit is 0.358uV.

DAStatus7_0 Register Address : 0x140

Field	Bits	Default	Type	Field Description
ts3_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the TS3 pin. When configured for thermistor measurement, the unit is 0.358uV.

DAStatus7_2 Register Address : 0x144

Field	Bits	Default	Type	Field Description
spi_mosi_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the SPI_MOSI pin. The unit is 0.358uV.

DAStatus7_3 Register Address: 0x148

Field	Bits	Default	Type	Field Description
dchg_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the DCHG pin. The unit is 0.358uV.

DAStatus7_4 Register Address : 0x14C

Field	Bits	Default	Type	Field Description
ddsg_counts	[23:0]	0x0	RO	Reports the 32-bit ADC counts for the most recent measurement on the DDSG pin. The unit is 0.358uV.

3.6 FET 控制

MOSFET 配置和状态读取

MOSFET 支持 AFE 控制和 MCU 控制，用户根据需求配置相关寄存器，用户如果完全仅 MCU 手动模式控制 MOS，仅需要在 Full Access 模式下发送 PCHGTEST、CHGTEST、PDSGTEST、DSGTEST 控制对应 MOS 的的开关，通过 FET Control 命令检测 MOS 的状态。

如果用户需要 AFE 参与到 MOS 的控制，无论是 AFE 自动控制还是 MCU+AFE 半自动控制，FET 均需要按照以下进行步骤进行配置：

1. fet_en 为 MOSFET 的总使能寄存器，通过 Manufacturing Register[fet_en]使能 MOS 总开关，也可以通过 FET_ENABLE 子命令（toggle 信号）控制 Manufacturing Register[fet_en]的信号，打开 AFE 控制 MOS 的总使能开关；

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

2. 在使能 MOS 总开关后，进一步在 FET_OPTIONS Register 配置 FET 的详细控制（建议默认值 0x0D），如果与用户有极低的功耗需求，在这里面也可以配置源跟随模式，在 sleep 下关闭 CHG PMUP 降低功耗；
3. 电荷泵可以提供 5.5V 或 11V 来驱动 CHG 和 DSG 引脚的栅极，用户可以通过修改 CHG_PUMP_CONTROL Register [LVEN] 中的配置位来设置所需的电压；
4. 如果用户使能了 PCHG 或者 PDSG，必须要配置退出预充和预防的条件，否则系统一直检测不到退出条件无法切换到正常的 CHG 或者 DSG 状态。
5. 配置完 MOS 基本功能后，用户可以配置是否选择 DFETOFF 和 CFETOFF 引脚联合控制 MOS，例如 DFETOFF Pin Config Register 配置为 0x98， CFETOFF Pin Config Register 配置为 0x9A，当检测到引脚输入为高时，MOS 正常打开，如果配置为低时，对应的 MOS 关闭。
6. 用户可以通过 FET Status 状态实时检测 MOS 是否有打开，或者配置 DCHG Pin Config 和 DDSG Pin Config 引脚为驱动输出信号（配置为 0x02），监测 MOS 的真实状态。
备注：相关寄存器描述在下页。

MOSFET 子命令

如果用户设置了 host_fet_en 使能，可以通过 FET 相关的子命令控制 MOSFET 的开关。FET 控制常用的子命令有两个，其中 FET Enable Register 命令是使能或者禁能 FET 总控制，该信号为 Toggle 信号，如果当前 fet_en 使能，发送该子命令后 fet_en 被禁能，若 fet_en 为禁能，发送该命令后 fet_en 使能。另一个命令为 FET Control Register，可以分别控制 PCHG、PDSG、CHG 和 DSG 信号，写 1 该 mos 禁能，写 0 该 mos 使能。建议使用 FET Control Register 命令自主控制 mosfet，不要多种命令搭配使用，例如不要同时使用 All FETs Off Register、FET Control Register 等不同命令控制 MOS，避免一些命令未清除导致 mos 一直被禁能。

MOSFET 未打开

用户按照上述配置和命令控制 FET 后，满足条件发现 MOSFET 未打开，需要查看 Safety Alert A/B/C, PF Alert A/B/C/D 的状态。其次通过子命令 FET Status Register 查询 mos 当前状态，确认 MOS 是真实未打开还是测量问题。如果用户设置了 PCHG 和 PDSG 使能，一定要配置退出 PCHG 和 PDSG 的条件。

PCHG 注意事项

如果客户需要使用 PCHG，需要设置 PRECHARGE_START_VOLTAGE 和 PRECHARGE_STOP_VOLTAGE Register 相关寄存器，到设置阈值进入 PCHG 或者从 PCHG 状态退出，如果写 0 则默认禁止该功能。若使用 PRECHARGE 功能，则一定配置 PRECHARGE_STOP_VOLTAGE Register，避免 AFE 一致处于 PCHG 状态不能切换至 CHG。

PDSG 注意事项

如果客户需要使用 PDSG，需要设置 PREDISCHARGE 相关寄存器，到设置时间或者阈值从 PDSG 状态退出，对应的寄存器为 PREDISCHARGE_TIMEOUT Register 和 PREDISCHARGE_STOP_DELTA Register。如果写 0 则默认禁止该功能，如果使能了 PDSG 功能但未配置退出条件，会导致 AFE 一直处于 PDSG 状态不能切换至 DSG。

放电时 CHG 自动打开

当大电流放电时，为了降低二极管的损耗 AFE 检测到放电电流大于 BODY_DIODE_THRESHOLD Register 阈值（默认值为 50mA），AFE 会自动开启 CHG 管，并关闭 PCHG 管子。

DDSG 和 DCHG PIN

DDSG 和 DCHG PIN 不仅可以做低边 MOS 的驱动信号，也可以用来作为高边 MOS 的状态信号。用户可以配置 DDSG Pin Config Register[ddsg_pin_fxn]为 2，作为 DSG MOS 的影子信号。当 DSG MOSFET 打开时，DDSG 引脚输出高电平，当 DSGFET 关闭时，DDSG 引脚输出为低电平。同理用户可以配置 DCHG Pin Config Register [dchg_pin_fxn]为 2，作为 CHG MOS 的影子信号。当 CHG MOSFET 打开时，DCHG 引脚输出高电平，当 CHG MOSFET 关闭时，DCHG 引脚输出低电平。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Manufacturing Register Address : 0x8013

Field	Bits	Default	Type	Field Description
fet_en	[4: 4]	0x0	RWHW	This bit enables or disables FET Test mode. In FET Test mode, the FET states are controlled by the FET Test subcommands. This is typically used during manufacturing to test FET circuitry. Note that safety checks still may force FETs off (or for body diode protection, on) in FET Test mode. 0 = Normal FET control is disabled. FET Test mode is enabled. Device will not turn on FETs unless FET Test subcommands instruct it to do so. 1 = Normal FET control is enabled. FET Test mode is disabled. Device will ignore FET Test subcommands.

FET_OPTIONS Register Address: 0x8824

Field	Bits	Default	Type	Field Description
fet_init_off	[5:5]	0x0	RW	0: Default host FET control state allows FETs to be on 1: Default host FET control state forces FETs off
pdsg_en	[4:4]	0x0	RW	0: PDSG FET is not used 1: PDSG FET is turned on before DSG
fet_ctrl_en	[3:3]	0x1	RW	0: FETs will not be turned on 1: FETs are controlled by the device
host_fet_en	[2:2]	0x1	RW	0: Host FET control commands are ignored 1: Host FET control commands are allowed
sleepchg	[1:1]	0x0	RW	0: CHG FET is turned off in SLEEP mode 1: CHG FET may be enabled in SLEEP mode
sfet	[0:0]	0x1	RW	0: Parallel FET mode: Body diode protection is disabled 1: Series FET mode: Body diode protection is enabled

CHG_PUMP_CONTROL Register Address: 0x8825

Field	Bits	Default	Type	Field Description
sleep_cpen	[3:3]	0x1	RW	0: Charge pumps for FET drivers are disabled in sleep mode 1: Charge pumps for FET drivers are enabled in sleep mode
sfmode_sleep	[2:2]	0x0	RW	0: Source-follower mode is not enabled on the DSG FET driver 1: Source-follower mode is enabled on the DSG FET driver while in SLEEP mode
lven	[1:1]	0x0	RW	This bit selects the charge pump overdrive level. 0: Charge pump low overdrive level (5.5 V) is selected 1: Charge pump high overdrive level (11 V) is selected
normal_cpen	[0:0]	0x1	RW	0: Charge pumps for FET drivers are disabled in normal mode 1: Charge pumps for FET drivers are enabled in normal mode

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

PRECHARGE_START_VOLTAGE Register Address : 0x8826

Field	Bits	Default	Type	Field Description
pchg_start_voltage	[15:0]	0x0	RW	PRECHARGE mode can be used to provide lower-current charging through the PCHG FET instead of the CHG FET for an under-voltage battery. When the minimum cell voltage is less than this threshold, PRECHARGE mode is activated. To disable PRECHARGE mode, set this value to 0.

PRECHARGE_STOP_VOLTAGE Register Address: 0x8828

Field	Bits	Default	Type	Field Description
pchg_stop_voltage	[15:0]	0x0	RW	PRECHARGE mode can be used to provide lower-current charging through the PCHG FET instead of the CHG FET for an under-voltage battery. When the minimum cell voltage is greater than or equal to this threshold, PRECHARGE mode is deactivated. To disable PRECHARGE mode, set this value to 0.

PREDISCHARGE_TIMEOUT Register Address: 0x882A

Field	Bits	Default	Type	Field Description
pdsg_timeout	[7:0]	0x5	RW	When PREDISCHARGE mode is enabled, the maxiumum duration of PREDISCHARGE mode can be set in 10-ms increments. 0: No timeout. Predischarge mode is exited when voltage conditions are met. All other values: Predischarge mode is exited and DSG is turned on after configured timeout.

PREDISCHARGE_STOP_DELTA Register Address: 0x882B

Field	Bits	Default	Type	Field Description
pdsg_stop_delta_en	[7:7]	0	RW	Bit[7]: 0: Predischarge voltage check disabled. Predischarge mode is exited for timeout. All other values:Predischarge mode exited when the LD pin voltage is measured within this delta of the top-of-stack voltage. Bit[6]: 0: Predischarge time out check disabled.
pdsg timeout en	[6:6]	0	RW	1: Predischarge mode is exited when reach preset pdsg timeout. [7:6]=00, cannot exit pre-charge; [7:6]=11, exit pre-charge as long as one of the conditions is met; [7:6]=01, exit pre-charge when Time out condition is met; [7:6]=10, exit pre-charge when LD voltage condition is met.
pdsg stop delta	[5:0]	0x32	RW	128mV/LSB. Predischarge mode exited when the LD pin voltage is measured within this delta of the top-of-stack voltage.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

FET Status Register Address : 0x7F

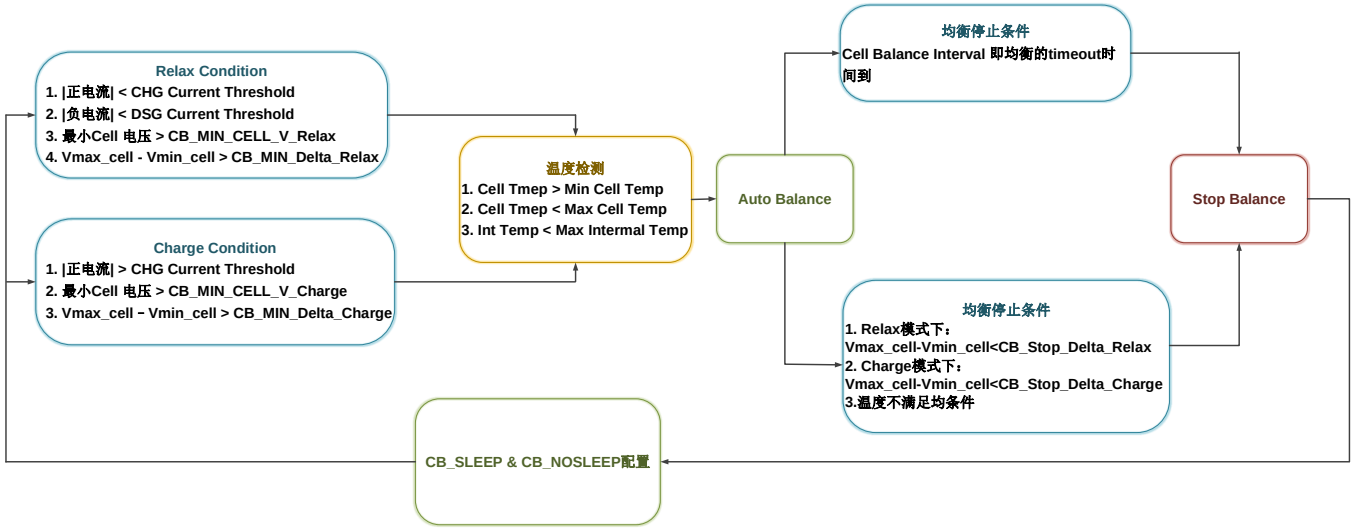
Field	Bits	Default	Type	Field Description
alrt_pin	[6:6]	0x0	RO	Indicates the status of the ALERT pin. 0 = The ALERT pin is not asserted. 1 = The ALERT pin is asserted.
ddsg_pin	[5:5]	0x0	RO	Indicates the status of the DDSG pin. 0 = The DDSG pin is not asserted. 1 = The DDSG pin is asserted.
dchg_pin	[4:4]	0x0	RO	Indicates the status of the DCHG pin. 0 = The DCHG pin is not asserted. 1 = The DCHG pin is asserted.
pdsg_fet	[3:3]	0x0	RO	Indicates the status of the PDSG FET. 0 = The PDSG FET is off. 1 = The PDSG FET is on
dsg_fet	[2:2]	0x0	RO	Indicates the status of the DSG FET. 0 = The DSG FET is off. 1 = The DSG FET is on.
pchg_fet	[1:1]	0x0	RO	Indicates the status of the PCHG FET. 0 = The PCHG FET is off. 1 = The PCHG FET is on.
chg_fet	[0:0]	0x0	RO	Indicates the status of the CHG FET. 0 = The CHG FET is off. 1 = The CHG FET is on.

3.7 均衡使用

3.7.1 自动均衡

GD30BM201X 芯片自动均衡采用奇偶均衡的方式，禁止相邻两个均衡电芯同时开启均衡，避免相连电芯同时均衡导致局部过热。下图是自动均衡逻辑，首先判断电池电压是否满足均衡进入的条件，满足后需要判断电芯温度和芯片内部温度，需要注意的是 TS1 引脚默认为热敏电阻输入，必须接入电阻确认 TS1 的读数在均衡范围要求内，如果条件均满足且没有告警事件发生，则会进入均衡状态。开始均衡后实时判断是否满足均衡退出条件或者均衡计数器时间到，任一满足其中 1 个则会停止均衡。例如最大电芯压差为 50mV，设置均衡计数器时间为 20S，期间如果电芯压差小于均衡停止压差，则会退出均衡。20S 后如果电芯压差仍大于 50mV 则再次进去均衡，如果小于均衡进入压差，则不满足均衡进入条件停止均衡。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.



用户在使用自动均衡功能时，建议按照以下步骤进行配置：

- 配置 0x8000 Power Config Register [loop_slow]为非 0 值，01/10/11 均支持；
- 打开均衡总开关，配置 0x8C00 BALANCING_CONFIGURATION[cb_en]为 1；
- 0x8C00 BALANCING_CONFIGURATION 中的[cb_nosleep] & [cb_sleep] & [cb_rlx] & [cb_chg] 可以控制是否允许在不同模式下开启均衡，建议 0x8C00 配置为 0xC3;
- 0x8C01 – 0x8C03 为均衡温度阈值寄存器，用户按照使用要求自行配置；
- 0x8C06 – 0x8C09 为充电模式均衡电压寄存器，0x8C0A – 0x8C0C 为空闲模式均衡电压寄存器用户按照使用要求自行配置；
- 0x8C04 CB_BALANCE_INTERVAL Register 为均衡时间计数器，默认 20S 均衡时间到后会再次判断均衡条件，如果满足会再次自动打开均衡；
- 配置均衡最大电芯节数 0x8C05 CB_BALANCE_MAX_CELLS，建议配置为 0x08；
- 配置充电电流阈值 CHG_CURRENT_THRESHOLD(0x8C14)和放电电流阈值 DSG_CURRENT_THRESHOLD (0x8C12)寄存器；
- 配置结束后退出 Config 模式，满足均衡条件后均衡会立即生效。

例如用户电池在充电、空闲状态运行均衡，BMS 通过比较实时电流与充放电电流阈值加工当前电池的状态，放电电流阈值寄存器为 DSG_CURRENT_THRESHOLD（默认值为 100userA），当放电电流大于放电电流阈值，AFE 认为处于放电状态停止均衡。充电电流阈值寄存器 CHG_CURRENT_THRESHOLD（默认值为 50userA），当充电电流大于充电电流阈值，AFE 认为充电状态。

在自动均衡模式下，GD30BM201x 带数据的子命令中可以实时自动更新均衡状态，用户读取 0x200 CB Active Cells Register 获取当前均衡的电芯，如果当前电芯在均衡，则对应的 bit 置 1，均衡停止后对应的 bit 置 0。注意在自动均衡模式下该寄存器只读，禁止写入。用户也可以读取 0x208 获取均衡总时间，也可以读取带数据的子命令 0x210 – 0x24C 和 0x15C 获取单节电芯的均衡时间，均衡计数器时间到后会更新均衡时间。

GD30BM2016 CB Active Cells Register：0x0200

23	22	21	20	19	18	17	16
RSVD_0	RSVD_0	RSVD_0	RSVD_0	RSVD_0	RSVD_0	RSVD_0	Cell16_CB
15	14	13	12	11	10	9	8
Cell15_CB	Cell14_CB	Cell13_CB	Cell12_CB	Cell11_CB	Cell10_CB	RSVD_0	Cell9_CB

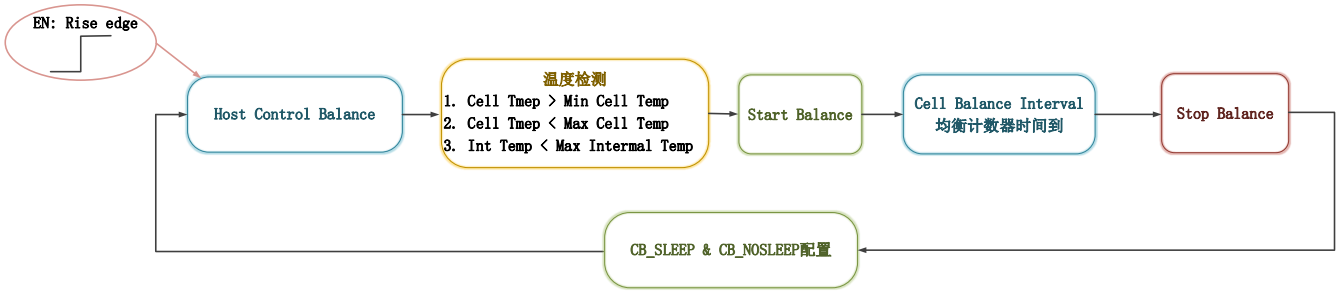
Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

7	6	5	4	3	2	1	0
Cell8_CB	Cell7_CB	Cell6_CB	Cell5_CB	Cell4_CB	Cell3_CB	Cell2_CB	Cell1_CB

3.7.2 手动均衡

GD30BM201X 也支持手动均衡，用户可以通过主机控制器完全手动控制均衡，对于主机均衡控制其方式更加灵活，用户加工出要均衡的电芯，发送特定的均衡子命令满足条件后则会开启均衡，手动均衡可以同时平衡相邻和非相邻的电芯这里与 TI 一致，但是建议用户在手动均衡时也采用奇偶均衡，避免相邻电芯同时均衡造成局部过热。

下图手动均衡逻辑，主机发送均衡子命令后，满足电芯温度和芯片内部温度条件，条件均满足且没有告警事件发生，则会进入均衡状态。开始均衡后实时判断是否满足均衡退出条件或者均衡计数器的时间到，任意满足其中 1 个则会停止均衡。如果需要再次均衡，则需要主机再次发送均衡子命令打开均衡，需要注意的是手动均衡的使能时对应 bit 的上升沿，所以需要先将均衡子命令清零后再写 1。



用户在使用手动均衡功能时，建议按照以下步骤进行配置：

- 配置 0x8000 Power Config Register [loop_slow]为非 0 值，01/10/11 均支持；
- 打开均衡总开关，配置 0x8C00 BALANCING_CONFIGURATION[cb_en]为 1；
- 0x8C00 BALANCING_CONFIGURATION 中的[3:0]可以控制是否允许在不同模式下开启均衡，仅手动均衡建议 0x8C00 配置为 0xC0;
- 0x8C04 CB_BALANCE_INTERVAL Register 为均衡计数器，阈值默认 20S，均衡时间到后关闭手动均衡；
- 配置均衡最大电芯节数 0x8C05 CB_BALANCE_MAX_CELLS，建议配置为 0x08；
- 配置结束后退出 setcfg 模式，等待寄存器生效；
- 写 0x200 CB Active Cells Register 对应的 bit 为 1，开启手动均衡；
- 均衡计数器时间到后，先将 0x200 CB Active Cells Register 清零，等待 500m 后再写对应的 bit 为 1，再次开启手动均衡。

在手动均衡模式下，GD30BM201x 带数据的子命令中仅用来控制均衡的打开与关闭，不能用来做电芯均衡状态指示。如果用户想了解当前均衡状态，也可以读取 0x208 获取均衡总时间，也可以读取 0x210 – 0x24C 和 0x15C 获取单节电芯的均衡时间，均衡计数器时间到后会更新均衡时间。

GD30BM2016 CB Active Cells Register： 0x0200

23	22	21	20	19	18	17	16
rsvd_0	rsvd_0	rsvd_0	rsvd_0	rsvd_0	rsvd_0	rsvd_0	cell16 cben
15	14	13	12	11	10	9	8
cell15 cben	cell14 cben	cell13 cben	cell12 cben	cell11 cben	cell10 cben	rsvd_0	cell9 cben
7	6	5	4	3	2	1	0

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

cell8 cben	cell7 cben	cell6 cben	cell5 cben	cell4 cben	cell3 cben	cell2 cben	cell1 cben

下面是 GD32BM2016 手动均衡的代码示例，输入参数是用户需要均衡的电芯，在代码中首先将电芯编号和均衡子命令的 Bit 做了移位对齐处理，在每次发送均衡命令之前先进行清零再写 1。另外为了避免相邻电芯均衡，在代码中加入奇偶掩码的处理，客户需要根据实际电芯连接情况，更改奇偶掩码确保不会同时打开相邻电芯的均衡，具体参考 GD30BM2010 CB Active Cells Register 定义。

均衡示例

```
#define MASK16_EVEN                    0x000154AA     // (0xAAAA& 0x1FF)|(0xAAAA >> 9) & 0x7F) << 10
#define MASK16_ODD                     0x000A955      // (0x5555& 0x1FF)|(0x5555 >> 9) & 0x7F) << 10
void bm2016_manual_cellbalance (uint16_t cell_number,cb_type_t cb_mode)
{
// Trigger manual equalization with the rising edge
    bm2016_subcommands_withdata(CELL_BALANCING_ACTIVE_CELLS, 0x000000, W4);
    delay_1ms(500);
    uint32_t cbcell_temp;
    /* low 9 bits (cells 1..9) + shifted high 7 bits (cells 10..16) */
    cbcell_temp = (cell_number & 0x01FFu)                    /* bits 0..8    -> 0..8 */
                | (((cell_number >> 9) & 0x7Fu) << 10);      /* bits 9..15 -> 10..16, bit9 = 0 */
    switch (cb_mode) {
            case CB_ODD:
                    /* handle odd balancing */
                cbcell_temp = cbcell_temp & MASK16_ODD;
              bm2016_subcommands_withdata(cbcell_temp, cbcell_temp, W4);
                    break;
            case CB_EVEN:
                    /* handle even balancing */
                    cbcell_temp = cbcell_temp & MASK16_EVEN;
              bm2016_subcommands_withdata(cbcell_temp, cbcell_temp, W4);
                    break;
            case CB_STOP:
                    /* handle stop balancing */
                  bm2016_subcommands_withdata(CELL_BALANCING_ACTIVE_CELLS, 0x000000, W4);
                    break;
            default:
                    bm2016_subcommands_withdata(CELL_BALANCING_ACTIVE_CELLS, 0x000000, W4);
                    break;
            }
}
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

3.8 保护功能

DSG/CHG 保护设置

保护生效分为以下步骤：

1. 首先设置保护条件，包括阈值、保护滤波时间、恢复时间等；具体的保护条件查看 Register Maps 中的 Protect 章节。
2. 其次需要使能保护寄存器：ENABLED_PROTECTIONS_A Register、ENABLED_PROTECTIONS_B Register、ENABLED_PROTECTIONS_C Register，这些寄存器中定义了具体的保护，例如过压、欠压、过流、过温等。
3. 如果用户要使用二次保护功能，配置也类似，先配置二次保护条件，在 ENABLED_PF_A Register、ENABLED_PF_B Register、ENABLED_PF_C Register、ENABLED_PF_D Register 中使能具体的故障保护，同时在 Manufacturing Register 和 PROTECTION_CONFIGURATION Register 选择是否是否使能永久失效保护和对应动作。
4. 完成上述步骤后，还需配置 CHG_FET_PROTECTIONS_A Registers、CHG_FET_PROTECTIONS_B Registers、CHG_FET_PROTECTIONS_C Registers、DSG_FET_PROTECTIONS_A 、DSG_FET_PROTECTIONS_B Register、DSG_FET_PROTECTIONS_C Register，才能将 MOS 控制与故障关联起来，完成上述步骤才能在发生故障时断开 MOS，实现保护功能。例如要使欠压保护生效，需要设置欠压保护条件 CUV_THRESHOLD、CUV_DELAY、CUV_RECOVERY_HYSTERESIS Register，然后使能 ENABLED_PROTECTIONS_A Register[cuv_prot_en]，并写 DSG_FET_PROTECTIONS_A Register[dsg_fet_cuv_en]为 1。
5. 注意 OCVL、OCDL、SCDL 中的 decrement delay time 设置值必须小于 recover time 设置值。

MOSFET 未断开

发生故障后对应的 MOSFET 未断开进行保护，用户直接子命令查询 Safety Alert A/B/C, PF Alert A/B/C/D 的状态。其次通过子命令 FET Status Register 查询 mos 当前状态，确认 MOS 是真实未断开还是测量问题。如果 MOS 未断开，查询保护条件、ENABLED_PROTECTIONSABC、CHG_FET_PROTECTIONSABC、DSG_FET_PROTECTIONS 的配置。

备注：其他寄存器具体描述请查看 Register Maps 中的 Protect 章节。

Manufacturing Register Address Offset: 0x8013

Field	Bits	Default	Type	Field Description
pf_en	[6: 6]	0x1	RWHW	This bit enables or disables Permanent Failure checks. Clearing this bit prevents Permanent Failure from triggering which is useful during manufacturing. 0 = Permanent Failure checks are disabled. 1 = Permanent Failure checks are enabled

PROTECTION_CONFIGURATION Register Address : 0x880A

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Field	Bits	Default	Type	Field Description
fetf_fuse	[8:8]	0x0	RW	When a Permanent Failure has been detected, the device may be configured to blow the fuse. This normally requires the voltage being above Min_Blow_Fuse_Voltage. When this bit is set and a FET failure occurs (CFETF or DFETF), the voltage requirement is bypassed. 0: Voltage must be above a threshold to blow the fuse when CFETF or DFETF trips. 1: If configured to blow the fuse and CFETF or DFETF occurs, fuse blow is attempted regardless of voltage.
pf_fets	[1:1]	0x1	RW	0: Permanent Failure does not cause the device to turn the FETs off. 1: Permanent Failure causes the device to turn the FETs off.

3.9 中断 ALERT 处理

当使用 ALERT 信号作为中断时候，当发生故障后芯片可以输出有效电平给到 MCU，触发故障中断给到 MCU，具体中断引脚配置可以参考引脚复用章节的介绍。针对中断使能相关的配置主要涉及到以下寄存器：

- POWER_CONFIG (0x8000) BIT14 配置为 1：可以用清除命令恢复 Alert 引脚电平。
- Alert Pin Config Register 需要配置为 2 = ALERT.
- DEFAULT_ALARM_MASK Register(0x881E)，选择 Alert 输出有效电平的故障源
- SF_ALERT_MASK_A (0x8815)
- SF_ALERT_MASK_B (0x8816)
- SF_ALERT_MASK_C (0x8817)
- PF_ALERT_MASK_A (0x8818)
- PF_ALERT_MASK_B (0x8819)
- PF_ALERT_MASK_C (0x881A)
- PF_ALERT_MASK_D (0x881B)

查询中断状态涉及直接命令 ALARM_STATUS (0x62)、ALARM_RAW_STATUS (0x64)。通过将 SF_ALERT 和 PF_ALERT 的对应比错误 mask 的 bit 置为 1，当发生对应终端后，ALERT 就会输出有效电平。用户可以通过查询 ALARM_STATSU 查询是哪些故障造成了 ALERT 事件。

当故障消失后，ALERT 信号重新回到无效状态。如果之前故障未消失，ALERT 信号仍继续断言，如果用户希望继续查询是否有其他故障发生，可以查询 ALARM_STATUS 和 ALARM_RAW_STATUS 来获知是否有新的中断发生。用户可以通过对 ALARM_STATUS 对应的 bit 写 1 的动作清除掉 ALARM_STATUS，ALARM_RAW_STATUS 只有故障消失才可以自动清零。

Power Config Register Address: 0x8000

Field	Bits	Default	Type	Field Description
normal_intr_mask	[14:14]	0x0	RW	0: enable normal intr; 1: mask normal intr，enable alarm assert

Alert Pin Config Register Address : 0x8202

Field	Bits	Default	Type	Field Description
alert_opt	[7: 2]	0x0	RW	The options for pin functions selected by these bit configurations are as follows:

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Field	Bits	Default	Type	Field Description
				OPT[5]: Polarity when configured as ALERT (no effect on GPO mode) 0 = Select active high 1 = Select active low OPT[4]: 0 = Digital output (OEN=0) 1 = No digital output (OEN=1, HZ) OPT[3]: 0 = Output high-level drive using REG1 1 = Output high-level drive using REG18 OPT[2]: 0 = Disable digital input (IE=0) 1 = Enable digital input (IE=1) OPT[1]: 0 = Disable digital input weak pull-up 1 = Enable digital input weak pull-up OPT[0]: 0 = Disable digital input weak pull-down 1 = Enable digital input weak pull-down
alert_pin_fxn	[1: 0]	0x0	RW	These bits configure which function this pin is used for: 0 = Unused 1 = General Output 2 = ALERT 3 = ADC Input or Thermistor

DEFAULT_ALARM_MASK Register Address : 0x881E

Field	Bits	Default	Type	Field Description
alm_en	[15:0]	0xf800	RWHW	[15] SSBC Setting this bit causes the Alarm Status()[SSBC] to be set and latched when Alarm Raw Status()[SSBC] is asserted. [14] SSA Setting this bit causes the Alarm Status()[SSA] to be set and latched when Alarm Raw Status()[SSA] is asserted. [13] PF Setting this bit causes the Alarm Status()[PF] to be set and latched when Alarm Raw Status()[PF] is asserted. [12] MSK_SFALERT Setting this bit causes the Alarm Status()[MSK_SFALERT] to be set and latched when Alarm Raw Status()[MSK_SFALERT] is asserted. [11] MSK_PFALERT Setting this bit causes the Alarm Status()[MSK_PFALERT] to be set and latched when Alarm Raw Status()[MSK_PFALERT] is asserted. [10] INITSTART Setting this bit causes the Alarm Status()[INITSTART] to be set and latched when Alarm Raw Status()[INITSTART] is asserted.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

				[9] INITCOMP Setting this bit causes the Alarm Status()[INITCOMP] to be set and latched when Alarm Raw Status()[INITCOMP] is asserted. [8] MSK_SFFAULT Setting this bit causes the Alarm Status()[MSK_SFALERT] to be set and latched when Alarm Raw Status()[MSK_SFALERT] is asserted. [7] FULLSCAN Setting this bit causes the Alarm Status()[FULLSCAN] to be set and latched when Alarm Raw Status()[FULLSCAN] is asserted. [6] XCHG Setting this bit causes the Alarm Status()[XCHG] to be set and latched when Alarm Raw Status()[XCHG] is asserted. [5] XDSG Setting this bit causes the Alarm Status()[XDSG] to be set and latched when Alarm Raw Status()[XDSG] is asserted. [4] SHUTV Setting this bit causes the Alarm Status()[SHUTV] to be set and latched when Alarm Raw Status()[SHUTV] is asserted. [3] FUSE Setting this bit causes the Alarm Status()[FUSE] to be set and latched when Alarm Raw Status()[FUSE] is asserted. [2] CB Setting this bit causes the Alarm Status()[CB] to be set and latched when Alarm Raw Status()[CB] is asserted. [1] ADSCAN Setting this bit causes the Alarm Status()[ADSCAN] to be set and latched when Alarm Raw Status()[ADSCAN] is asserted. [0] WAKE Setting this bit causes the Alarm Status()[WAKE] to be set and latched when Alarm Raw Status()[WAKE] is asserted.

ALARM_STATUS 和 ALARM_RAW_STAUS 包含了多种故障汇总信息。

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
SSBC	SSA	PF	MASK_SFALERT	MSK_PFALERT	INITSTART	INITCOMP	RSV
FULLSCAN	XCHG	XDSG	SHUTV	FUSE	CB	ADSCAN	WAKE

例如 SSBC 对用 SF_ALERT_B 和 SF_ALAERTC 等事件汇总 BIT, SSA 是 SF_ALAERTA 事件的汇总 BIT，PF 是所有 PF_ALERT 汇总 BIT。当查询到这些 BIT 为 1 时，用户可以进一步查询 SF_ALAERT_A/B/C, PF_ALERT_A/BC/D 来查询具体发生故障的原因。

3.10 WatchDog 功能使用

BM2016 不仅可以用 ALER 指示发生了看门狗超时事件，还增加了 DCHG 引脚用来监测看门狗超时事件发生，使能 watchdog 功能后，当在规定时间内没有接受到通信的指令，AFE 认为发生了看门狗超时故障后，DCHC 的信号从低翻转为高，MCU 监测到该信号后可以灵活的复位 AFE，保证系统更安全的运行。具体 WatchDog 相关的配置主要涉及到以下寄存器：

- POWER_CONFIG (0x8000) BIT[5:4], 写 1 配置 mcu wdg timeout;
- WDG EN Register (0x804C)BIT[1]写 1 使能看门狗功能；
- HWD_DELAY Register(0x8876)配置看门狗 timeout 时间
- ENABLED_PROTECTIONS_C Register（0x880E)BIT[1]使能看门狗保护；

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

- 对应在 CHG_FET_PROTECTIONS_C(0x8811), DSG_FET_PROTECTIONS_C Register(0x8814） 关联 MOS 保护
- 0x8E2A bit[0]配置为 1，断开 DCHG 影子输出信号，DCHG Pin Config Register （0x8207）配置为 0x80，复用为看门狗输出信号

Power Config Register Address: 0x8000

Field	Bits	Default	Type	Field Description
mcu_wdg_tim	[5: 4]	0x0	RW	mcu wdg timeout

WDG EN Register Address : 0x804C

Field	Bits	Default	Type	Field Description
mcu_wdg_en	[1: 1]	0x0	RW	mcu wdg enable

HWD_DELAY Register Address : 0x8876

Field	Bits	Default	Type	Field Description
hwd_delay	[15:0]	0x3c	RW	This parameter configures the Host Watchdog timeout. If communications are not received for this many seconds, the Host Watchdog Fault is triggered.

DCHG Pin Config Register Address : 0x8207

Field	Bits	Default	Type	Field Description
dchg_opt	[7: 2]	0x0	RW	The options for pin functions selected by these bit configurations are as follows: When 0x8e2a[0] = 0, DCHG acts as the shadow signal of CHG: OPT[5]: --- 0: Low-side CHG function, output is low when CHG is enabled; --- 1: Low-side CHG function, output is high when CHG is enabled; When 0x8e2a[0] = 1, DCHG acts as a regular GPIO or analog mapping: Multi-function Pin Configuration: OPT[5], OPT[4],OPT[3] 000: Default CHG function, default low. 100: Watchdog multiplexed output (reset the host). 101: General digital output (can be configured to use REG1 LDO or REG18 LDO for drive power supply, can be configured to weak pull-down to VSS or weak pull-up to REG1). 110: DCHG function (fault-related signal, can be used to control external protection circuits, used for interrupt control when the host manages CFETOFF to turn off CHGFET; configurable in NORMAL, SLEEP, DEEPSLEEP modes). 111: ADC or Tsensor input (unused for the 64-pin package). Additional Configurations: OPT[3]: 0 = Output high-level drive using REG1.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Field	Bits	Default	Type	Field Description
				1 = Output high-level drive using REG18. OPT[2]: 0 = Disable digital input (IE=0). 1 = Enable digital input (IE=1). OPT[1]: 0 = Disable weak pull-up for digital input. 1 = Enable weak pull-up for digital input. OPT[0]: 0 = Disable weak pull-down for digital input. 1 = Enable weak pull-down for digital input. When selecting pins for ADCIN or thermistor functions, OPT[5:0] bits are used as follows:** OPT[5:4]: If unused, set OPT[4] to 1. OPT[3]: Is the pin used as ADC input 0 = No 1 = Yes OPT[2]: Is the pin used as Tsensor input with pull-up resistor? 0 = No 1 = Yes OPT[1]: 0 = Not used for FET temperature protection. 1 = Used for FET temperature protection. OPT[0]: 0 = Not used for cell temperature protection. 1 = Used for cell temperature protection.
dchg_pin_fxn	[1: 0]	0x0	RW	These bits configure which function this pin is used for: 0 = Watchdog timer reset output. 1 = General Output. 2 = DCHG. 3 = ADC Input or Thermistor.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

4 初始化流程

器件的初始化主要是对于一些重要参数进行配置，其中包含系统参数（MOSFET 控制，采样速率，低功耗模式等），校准（如采样电阻值），引脚重新映射（NTC，GPIO 等），一次和二次保护参数以及均衡等。器件初始化流程主要包括 6 部分，具体如下：

1. 将器件进入 seal 模式，首先 unseal 后再进入 full access，进入 set_cfgupdate，此时才可以对于寄存器进行操作。
2. 系统寄存器配置，寄存器基地址 0x80xx，查看寄存器表 sysc 的部分，主要用来设置系统低功耗模式，配置 deepsleep，shutdown，sleep 模式下电荷泵，MOSFET 控制，时钟等参数。设备内部 LDO，REG0，REG1，REG2 的使能与电压, MOSFET 控制方式。
3. 保护相关参数，寄存器基地址 0x88xx，查看寄存器表格 protect 的部分，主要包含 cell mode（串数），COV，CUV，OCC，OCD1，OCD2，SCD，OTC，OTD，UTC，UTD 等相关一次保护，以及对应的二次保护参数，部分 MOSFET 保护的管关联控制。
4. 引脚映射，寄存器基地址 0x82xx，查看寄存器表格 pad 的部分，主要涉及到 TSx 的（上拉电阻和使能），CFETOFF，DFETOFF，DDSG，DCHG，SPI_MOSI，ALERT 等引脚的 GPIO 或者 NTC 的配置状态。
5. 均衡相关的参数，寄存器基地址 0x8Cxx，查看寄存器表格 cb 的部分，主要包含均衡使能，均衡个数，均衡最小电压，均衡最小差值等。
6. 校准参数，寄存器基地址 0x84xx，查看寄存器表格 adc 的部分。

需要注意的是 GD30BM2016 大部分寄存器都已经在工厂实现了校准，不需要用户做校准。采样电阻是客户自行根据电流选择的，需要设置 ccgain。上拉电阻温度系数需要和 NTC 类型和组织进行相应的配置，这个部分会推荐默认值，客户初始化自行加载。以上参数配置没有顺序要求，配置完这些参数后，直接执行 exit_cfgupdate，配置就可以生效。

4.1 初始化配置示例

初始化示例代码使用 python 编写，展示了 I2C 读写寄存器，其中写函数定义为 BM2016.Memory_write (command/reg_add, tx_data,len)，用户可参考下面示例来配置 GD30BM2016。

4.1.1 CONFIG_UPDATE 模式

GD30BM2016 默认为 SEALED 模式，该模式下主机可以读取大部分告警和数据，无法直接读取或更改数据寄存器的设置。如果想读取或者更改所有寄存器的设置，需要进入 UNSEALED 模式后再进入 FULLACCESS 模式，CONFIG_UPDATE 模式仅在 FULLACCESS 模式下可用。为了更改设置寄存器，用户需要在初始化开始进入 CONFIG_UPDATE 模式。

```
BM2016.write(0x3E, 0x032C, width=2)  # 进入 seal mode
BM2016.Memory_write(0x0280, 0x5A5A5A5A, 4)   #Security_access
BM2016.Memory_write(0x0280, 0xA5A5A5A5, 4)
BM2016.write(0x3E, 0x0338, width=2)    # 进入 config update mode
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

4.1.2 保护寄存器配置

保护寄存器包含了电池串数、COV，CUV，OCC，OCD1，OCD2，SCD，OTC，OTD，UTC，UTD 等相关一次保护，以及对应的二次保护参数，部分 MOSFET 保护的管关联控制。以下是保护寄存器的示例配置，供用户参考。

```
BM2016.Memory_write(0x8808 ,0xFEFF, 2)      #Cell mode,  FEFF 对应 16 串模式
BM2016.Memory_write(0x880A ,0x0602, 2)
BM2016.Memory_write(0x880C ,0xFC, 1)        #Enable Protection A, Enable SCD, OCD1,
OCD2, OCC, CUV, COV
BM2016.Memory_write(0x880D ,0Xf3, 1)        #Enable Protection B, Enable OTC, OTINT,
OTD, OTC, UTD, UTC. UTINT disable
BM2016.Memory_write(0x880E ,0xE8, 1)        #Enable Protection C, OCD3, SCDL, OCDL
ENBALE, COVL, PTO and WATCHDOG disable
BM2016.Memory_write(0x881C ,0x3E8, 2)       #BODY DIODE TH 1A
BM2016.MCU_OTP_write(0x8824 ,0x0D, 1)       #FET 受控于 芯片自身，使能 BODY DIODE 保护
BM2016.MCU_OTP_write(0x8825 ,0x09, 1)       #CP ON when sleep, normal and CP 电压 11V

BM2016.Memory_write(0x882C ,0x0088, 2)      #CUV 阈值 设置为 2.72V, step 为 20mV
BM2016.Memory_write(0x882E ,0x0400, 2)      #CUV 阈值延迟 设置为 4S,  step 为 3.9ms
BM2016.Memory_write(0x8830 ,0x0B, 2)       #CUV 恢复阈值 3V，0.55V 的迟滞 20mv step

BM2016.Memory_write(0x8832 ,0x00D1, 2)      #COV 阈值调整，过压阈值设置为 4.2V
BM2016.Memory_write(0x8834 ,0x02D5, 2)      #COV 阈值延迟 设置为 3S
BM2016.Memory_write(0x8836 ,0x02, 1)        #COV 阈值 4.1V 的迟滞 20mv step
BM2016.Memory_write(0x8837 ,0x01, 1)        #COVL limit 设置为 1
BM2016.Memory_write(0x8838 ,0x02, 1)        #COVL DEC 时间设置为 2S
BM2016.Memory_write(0x8839 ,0x04, 1)        #C0VL 恢复时间设置为 4S

BM2016.Memory_write(0x883A ,0x0001, 2)      #OCC 配置，2A 过流充电保护，采样电阻 1mohm,
(N-1)*2mv/采样电阻值
BM2016.Memory_write(0x883C ,0x64, 1)        #OCC 延迟保护设置为 400mS
BM2016.Memory_write(0x8840 ,0x00C8, 2)      #OCC 保护恢复阈值，单位 mA，1.8A

BM2016.Memory_write(0x8844 ,0x0009, 2)      #OCD1 配置，18mv 过放电保护，采样电阻 1mohm
BM2016.Memory_write(0x8846 ,0x64, 1)        #OCD1 延迟保护设置为 400mS
BM2016.Memory_write(0x8848 ,0x000F, 2)      #OCD2 配置，30mV 过放电保护，采样电阻 1mohm
BM2016.Memory_write(0x884A ,0x64, 1)        #OCD2 延迟保护设置为 400mS
BM2016.Memory_write(0x8852 ,0xFF38, 2)      #OCD 恢复阈值，设置为-200mA，过载消失后，超过恢
复时间就恢复

BM2016.Memory_write(0x884B ,0x02, 1)       #SCD 配置，40mv 过放电保护，采样电阻 1mohm，电
流值为 16A
BM2016.Memory_write(0x884C ,0x0F, 1)        #SCD 延迟保护设置为 450us, step 31.25us
BM2016.Memory_write(0x884D ,0x06, 2)        #SCD 短路恢复 6S 如果负载移除后，Step 1S
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
BM2016.Memory_write(0x884E ,0x1F40, 2)       #OCD3 阈值 单位 userA,
BM2016.Memory_write(0x8850 ,0x02, 1)         #OCD3 延迟滤波 2S

BM2016.Memory_write(0x8854 ,0x00, 1)         #OCDL LATCH LIMIT 为 0
BM2016.Memory_write(0x8855 ,0x0A, 1)         #OCDL DEC DEALY 为 10S
BM2016.Memory_write(0x8856 ,0x01, 1)         #OCDL RECOVER TIME 为 1S
BM2016.Memory_write(0x8858 ,0x64, 1)         #OCDL RECOVER CHAGRE TH 为 100mA

BM2016.Memory_write(0x885A ,0x00, 1)        #SCDl LATCH LIMIT, 0 次
BM2016.Memory_write(0x885B ,0x0A, 1)         #SCDl DEC DEALY 10S
BM2016.Memory_write(0x885C ,0x0A, 1)         #SCDl RECOVER TIME 10S
BM2016.Memory_write(0x885E ,0x00C8, 2)       #SCDl RECOVER CHAGRE TH 200mA

BM2016.Memory_write(0x8860 ,0x35, 1)        #OTC 配置, 充电过温阈值 49℃
BM2016.Memory_write(0x8861 ,0x03, 1)         #OTC delay 配置时间为 3S
BM2016.Memory_write(0x8862 ,0x31, 1)        #OTC 恢复配置阈值 68℃
BM2016.Memory_write(0x8863,0x44, 1 )         #OTD 配置, 充电过温阈值 63℃
BM2016.Memory_write(0x8864 ,0x03, 1)        #OTD delay 配置时间为 3S
BM2016.Memory_write(0x8865 ,0x3F, 1)         #OTD 配置, 充电过温阈值 50℃

BM2016.Memory_write(0x886C ,0xFB, 1)        #UTC 配置, 充电低温阈值 -5℃
BM2016.Memory_write(0x886D ,0x02, 1)         #UTC delay 配置时间为 2S
BM2016.Memory_write(0x886E ,0x01, 1)        #UTC 恢复配置阈值 1℃
BM2016.Memory_write(0x886F,0xEE, 1)         #UTD 配置, 放电过温阈值 -18℃
BM2016.Memory_write(0x8870,0x03, 1)         #UTD delay 配置时间为 3S
BM2016.Memory_write(0x8871,0xF1, 1)         #UTC 恢复配置阈值-15℃

BM2016.Memory_write(0x8872,0xEC, 1)         #UTD delay 配置时间为 2S
BM2016.Memory_write(0x8873,0x02, 1)         #UTC 恢复配阈值 -20℃

BM2016.Memory_write(0x8875,0x03, 1)         #RECOVERY DELAY 3S
```

4.1.3 引脚复用功能

GD30BM2016 器件设定了多个多功能引脚。多功能管脚包括 TS1,TS2, TS3, CFETOFF, DFETOFF, ALERT, SPI_MOSI,SPI_CS,DCHG, DDSG，每个引脚可以独立配置成不同的工作模式。下面是 DDSG 和 TSX 复用案例，用户根据实际复用情况进行配置。Pin 复用寄存器基地址 0x8200，具体地址请查看寄存器表 pad 的 sheet。

```
BM2016.Memory_write(0x8202 ,0x5B, 1)         #alert 作为告警信号，高电平有效
BM2016.Memory_write(0x8207 ,0x53, 1)         #DCHG 作为温度检测，只是测量，不保护
BM2016.Memory_write(0x8208 ,0x53, 1)         #DDSG 作为温度检测，只是测量，不保护
BM2016.Memory_write(0x8206 ,0x5B, 1)         #MOSI-MOSI 作为温度检测，MOSFET 保护
BM2016.Memory_write(0x8209 ,0x50, 1)         #TS1 作为 Tsensor，作为电芯保护
BM2016.Memory_write(0x820a ,0x50, 1)         #TS3 作为 Tsensor，作为电芯保护
BM2016.Memory_write(0x8210, 0x90, 1)          #PAD CFG 配置，TS 18K 上拉
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

4.1.4 ADC 配置

ADC 寄存器主要是配置电流电压通道的 gain 和 offset、温度系数等，GD30BM2016 大部分增益和 offset 等已经在工厂实现了校准，不需要用户另外再校准和配置。但电流采样电阻、温度 NTC 等是客户自行选择的，需要配置相应的寄存器，以下给出了 ADC 配置的简单示例。

```
BM2016.Memory_write(0x842C ,0x7530, 2)          #1mohm 采样电阻，  寄存器值 = 30000/R_sense
BM2016.Memory_write(0x8448 ,0XEC67, 2)         # 18K 上拉 A1 系数
BM2016.Memory_write(0x844A ,0X4262, 2)          #18K 上拉 A2 系数
BM2016.Memory_write(0x844C ,0XF6D7, 2)          #18K 上拉 A3 系数
BM2016.Memory_write(0x844E ,0X04C4, 2)          #18K 上拉 A4 系数
```

4.1.5 系统寄存器配置

系统寄存器包含了系统模式、LDO 设置、MOSFET 控制、时钟等参数，用户需根据自身需求，配置相应的寄存器。寄存器基地址 0x8000，具体地址请查看寄存器表 sysc 的 sheet。

```
BM2016.Memory_write(0x8000 ,0x4F86, 2)     #LOOP 降速 1/2 支持均衡，内部中断使能
BM2016.Memory_write(0x8002 ,0x13, 1)       #REG1 和 REG2 打开，REG1 使能 5V，REG2 3.3V
BM2016.Memory_write(0x8003 ,0x01, 1)       # PRE-REGULATOR 开
BM2016.Memory_write(0x8008 ,0x0A, 1)       # COMM IDLE TIME 10S
BM2016.Memory_write(0x8013, 0x50, 1)      #如果无告警就打开充放电 MOS FET
BM2016.Memory_write(0x801a, 0x1F40, 2)    #sleep chg threshold 设置为 64V，避免睡眠不能充电
```

4.1.6 均衡寄存器配置

GD30BM2016 支持电芯之间的均衡，用户可自行配置均衡开启和关闭的条件实现自动均衡，另外用户也可以通过主机发送命令实现手动均衡。以下为电芯均衡示例，主要包括均衡电压、电流、温度等。

```
BM2016.Memory_write(0x8C00 ,0xC3 , 1)    #CB 开关 ENABLE，CB 静置均衡和 CB 充电均衡开启
BM2016.Memory_write(0x8C01 ,0xEC , 1)    #min cell temp -20℃
BM2016.Memory_write(0x8C02 ,0x3C , 1)    #max cell temp 60℃
BM2016.Memory_write(0x8C03 ,0x46 , 1)    #max int temp 70℃
BM2016.Memory_write(0x8C04 ,0x05 , 1)    #max charge timer 5s
BM2016.Memory_write(0x8C05 ,0x08 , 1)    #max  balance cell 8
BM2016.Memory_write(0x8C06 ,0x0E42 , 2)   # 充电均衡 最小电压 3650mv
BM2016.Memory_write(0x8C08 ,0x28 , 1)   # 充电均衡开始条件 最小 cell 和最大 cell 电压 40mv
BM2016.Memory_write(0x8C09 ,0x14 , 1)   #充电均衡结束条件最小 cell 和最大 cell 电压 20mv
BM2016.Memory_write(0x8C0A ,0x0E42 , 2)   #静置均衡 最小电压 3650mv
BM2016.Memory_write(0x8C0C ,0x28 , 1)   # 静置均衡开始条件 最小 cell 和最大 cell 电压 40mv
BM2016.Memory_write(0x8C0D ,0x14 , 1)   # 静置均衡结束条件 最小 cell 和最大 cell 电压 20mv
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

4.1.7 初始化配置生效

以上寄存器的均在 CONFIG_UPDATE 模式下更改，用户在更改完成后，主机必须发送 EXIT_CFG_UPDATE 命令退出该模式，退出后器件根据最新的配置正常运行，执行所有测量和保护监控，EXIT_CFG_UPDATE 命令如下图所示。

```
BM2016.write(0x3E, 0x033C, width=2)    #退出 config_update
```

4.2 采样数据和寄存器状态读取

执行完初始化流程后，GD30BM2016 进入正常工作模式，对电池包电压、电流、状态进行监控，同时检测到电池异常执行保护动作。用户通过发送 dircmd 命令，读取常规的 cell 电压、电流、温度、告警等信息。Indir cmd with data 这部分主要是一些电压，电流和温度等信息的中间 ADC 原始码值，用户使用这些码值自行计算一些校准参数，用于工厂产线校准。

另外用户也可发送子命令控制器件，主要是是对于一些操作的一键直达。例如用户可以发送模式切换、MOSFET 直接控制、复位、故障模式退出等常用功能。

```
#-------- 读取 cell 电压值-------------------
CELL1 = BM2016.read(0x14, 2)
CELL2 = BM2016.read(0x16, 2)
CELL3 = BM2016.read(0x18, 2)
CELL4 = BM2016.read(0x1A, 2)
CELL5 = BM2016.read(0x1C, 2)
CELL6 = BM2016.read(0x1E, 2)
CELL7 = BM2016.read(0x20, 2)
CELL8 = BM2016.read(0x22, 2)
CELL9 = BM2016.read(0x24, 2)
CELL10 = BM2016.read(0x28, 2)
CELL11 = BM2016.read(0x2A, 2)
CELL12 = BM2016.read(0x2C, 2)
CELL13 = BM2016.read(0x2E, 2)
CELL14 = BM2016.read(0x30, 2)
CELL15 = BM2016.read(0x32, 2)
CELL16 = BM2016.read(0x3C, 2)
# -------- 读取充放电电流值-------------------
cc2_current = BM2016.read(0x3A, 2)
if cc2_current > 32768:
    cc2_current = 65536 - cc2_current
    print('cell discharge current Unit with mA')
else:
    print('cell charge current Unit with mA')
print(cc2_current)
#-------- 读取温度值-------------------
TS1 = BM2016.read(0x70, 2)
TS3 = BM2016.read(0x74, 2)
MOSI_temp = BM2016.read(0x76, 2)
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
print('TS temperature TS1, TS2, MOSI_TEMP Unit with DegC')
print(TS1*0.1,TS3*0.1,MOSI_temp*0.1)
```

4.3 采样数据转换计算

电芯电压、电池包电压等测量值计算、电流、温度码值转换公式在本章节中列出，其中 CC2 回读结果为 userA，系统默认 userA 为 1mA。如果需要修改 user A 参考 protect 寄存器中的 0x8807 DA_CONFIGURATION [1:0]。

- CELL_VOLTAGE 计算：

$$V_{step} = \frac{5 * V_{REF1}}{2^{23}} \approx \frac{5 * 1.212V}{2^{23}} \approx 0.722uV$$

$$V_{cell} = Count_{cell\ voltage} * V_{step} \approx Count_{cell\ voltage} * \ 0.722uV$$

- I_VOLTAGE 计算：

$$I_{step} = \frac{V_{REF2}}{5 * 2^{23}} \approx \frac{1.24V}{5 * 2^{23}} \approx 28.896nV$$

$$V_{current} = Count_{current} * I_{step} \approx Count_{current} * 28.896nV$$

- CC2 电流计算：

$$Current_{cc2} = CC2_Current * userA$$

STACK、PACK、LD VOLTAGE 回读结果为 userV，系统默认 userV 为 8mV，用户也可以通过 protect 寄存器中的 0x8807 DA_CONFIGURATION [2]设置 userV 的单位。

- STACK_VOLTAGE 计算：

$$V_{stack} = Count_{stack} * \mathrm{user}V$$

- PACK_VOLTAGE 计算：

$$V_{pack} = Count_{pack} * \mathrm{user}V$$

- LD_VOLTAGE 计算：

$$V_{LD} = Count_{LD} * \mathrm{user}V$$

温度读取值使用 DIRECT_COMMAND，所有温度的回读单位是 0.1℃，转换关系为：

$$temp = Count_{temp} * 0.1℃$$

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

5 通信函数封装

GD30BM2016 器件支持直接命令、子命令以及读取和写入 RAM 寄存器，采用 SPI 或 I2C 发送通信数据包时，器件可识别的命令 ID 和寄存器地址均一致，只是选择通信接口不同，本报告将提供基于 I2C 接口的通信数据包和序列示例，同时会提供 GD30BM2016 的头文件，包含直接命令、子命令、寄存器地址的宏定义，用户可以直接在代码中用宏定义替换详细的命令 ID 或者寄存器地址。

5.1 直接命令函数

下面为 I2C 的直接命令格式，其中：S 为 start 位， RS 为 restart 位，SLV_ADDR 代表 slave address 即 GD30BM2016 地址，W 为读写位中的写，A 为应答位表示主机或者从机 ack，N 为应答位表示主机或者从机 no ack，DirCMD 是直接命令 ID，BYTEx 是有效数据 Byte，P 为 stop 位。

I2C 写指令：

S	SLV_ADDR	W	A	Dir CMD	A	BYTE0	A	...	BYTEn	A	CRC8	N	P

I2C 读指令：

S	SLV_ADDR	W	A	Dir CMD	A	RS	SLV_ADDR	W	A	BYTE0	A	...	BYTEn	A	CRC8

直接命令即向器件直接写入直接命令 ID，以用来写入数据、触发动作或者读取器件数据。在器件配置好后，常用直接命令监控器件的电压、电流、温度等信息。下面展示了直接命令的读写代码示例，具体的直接命令表查阅 GD30BM2016 软件用户指南或者 GD30BM2016 的头文件。

直接命令读写代码示例

```
/*!
    \brief        write or read gd30bm2016 direct commands
    \param[in]    command: gd30bm2016 direct command
                  data: write data for direct command
                  type: read/write Type
    \param[out] none
    \retval       none
*/
void bm2016_direct_commands(uint8_t command, uint16_t data, trans_type type)
{
    //type: R = read, W = write
    uint8_t tx_data[2] = {0x00, 0x00};

    //little endian format
    tx_data[0] = data & 0xff;
    tx_data[1] = (data >> 8) & 0xff;

    if(type == R){
        read_reg(command, 2);
    }
    else {
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
            write_reg(command, tx_data, 2);
        }
        delay_1ms(1);
}
```

5.2 不带数据子命令函数

下面为不带数据子命令格式示例，蓝色为 MCU 发出的指令，白色为从机 GD30BM2016 应答，0x3E 为子命令入口地址，subcmd_l 为子命令低字节，subcmd_h 为子命令高字节，CRC8 为校验和，用户可以发送不带数据的子命令控制器件。

I2C 写

S	SLV_ADDR	W	A	0x3E	A	subcmd_l	A	subcmd_h	A	CRC8	N	P

子命令操作向 0x3E/0X3F 入口地址写对应的子命令，例如向 0x3E（低字节）和 0x3F（高字节）写入 0x0304 命令，GD30BM2016 进入 DEEPSLEEP，子命令 ID 与 BQ76952 不同，用户严格按照下表发送子命令 ID。具体的子命令表查阅 GD30BM2016 软件用户指南或者 GD30BM2016 的头文件。

不带数据子命令写代码示例

```
/*!
    \brief        write gd30bm2016 only Subcommands
    \param[in]    command: gd30bm2016 Subcommands
    \param[out] none
    \retval       none
*/
void bm2016_only_subcommands(uint16_t command)
{
    uint8_t tx_reg[2] = {0x00, 0x00};
    //tx_reg in little endian format
    tx_reg[0] = command & 0xff;
    tx_reg[1] = (command >> 8) & 0xff;
    write_reg(0x3E,tx_reg,2);
    delay_1ms(1);
}
```

5.3 带数据子命令函数

带数据的子命令格式如下所示，读写格式与 BQ76952 不同，用户需按照该格式发送带数据的子命令控制器件。首先将 16 位子命令地址写入 0x3E（低位字节）和 0x3F（高位字节），并将现有数据自动填充到 32 字节的传输缓冲区 0x40–0x5F。将该数据的校验和数据长度写入地址 0x60。checksum 和 len 都只针对需要读写的数据。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

I2C 写

S	SLV_ADDR	W	A	0x3E	A	subcmd_l	A	subcmd_h	A	CRC8	N	P		
S	SLV_ADDR	W	A	0x60	A	checksum	A	len(W)	A	CRC8	N	P		
S	SLV_ADDR	W	A	0x40	A	BYTE0	A	...	A	BYTEn	A	CRC8	N	P

I2C 读

S	SLV_ADDR	W	A	0x3E	A	subcmd_l	A	subcmd_h	A	CRC8	N	P
S	SLV_ADDR	W	A	0x61	A	len(R)	A	CRC8	A	P		

S	SLV_ADDR	W	A	0x40	A	RS	SLV_ADDR	R	A	BYTE0	A	...	BYTEn	A	CRC8	N	P
S	SLV_ADDR	W	A	0x60	A	RS	SLV_ADDR	R	A	BYTE0	A	...	BYTEn	A	CRC8	N	P

带数据的子命令表包括读和写命令，读命令通过将子命令 ID 写入命令地址 0x3E 中来读取器件型号。将读取长度写入 0x61 寄存器中，然后从地址 0x40 的数据缓冲区中读取数据，从地址 0x60 的数据缓冲区读取校验和长度。写命令先向 0x3E 入口地址写入子命令 ID，0x60 寄存器中写入 crc 校验值和数据长度，最后在缓冲寄存器 0x40-0x5F 写入数据，具体的子命令表查阅 GD30BM2016 软件用户指南或者 GD30BM2016 的头文件。另外写入大于两个长度以上的字节建议调用寄存器读写代码。

带数据子命令读写代码示例

```
/*!
    \brief        write or read gd30bm2016 subcommands with data
    \param[in]    command: gd30bm2016 subcommands
                  data: write data for subcommands
                  type: read/write Type
    \param[out] none
    \retval       none
*/
void bm2016_subcommands_withdata(uint16_t command, uint32_t data, trans_type type)
{
    uint8_t tx_reg[6] = {0};
    uint8_t tx_buffer[2] = {0};

    //TX_Reg in little endian format
    tx_reg[0] = command & 0xff;
    tx_reg[1] = (command >> 8) & 0xff;
  tx_reg[2] = data&0xff;
    switch(type){
        case R:
          write_reg(0x3E, tx_reg, 2);
            delay_1ms(1);
            tx_buffer[0] = 4; //combined length of register address and data
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
                write_reg(0x61, tx_buffer, 1); // Write the checksum and length
                delay_1ms(1);
                read_reg(0x40, 4); // Write the checksum and length
                delay_1ms(1);
            break;
            case W:
                write_reg(0x3E, tx_reg, 2);
                tx_buffer[0] = checksum_calculate(&tx_reg[2], 1);
                delay_1ms(1);
                tx_buffer[1] = 0x81; //combined length of register address and data
        write_reg(0x60, tx_buffer, 2); // Write the checksum and length
                delay_1ms(1);
              write_reg(0x40, &tx_reg[2], 1); // Write the checksum and length
                delay_1ms(1);
            break;
    case W2:
                tx_reg[3] = (data >> 8)&0xff;
                write_reg(0x3E, tx_reg, 2);
                delay_1ms(1);
                tx_buffer[0] = checksum_calculate(&tx_reg[2], 2);
                tx_buffer[1] = 0x82; //combined length of register address and data
        write_reg(0x60, tx_buffer, 2); // Write the checksum and length
                delay_1ms(1);
              write_reg(0x40, &tx_reg[2], 2); // Write the checksum and length
                delay_1ms(1);
            break;
            case W4:
                tx_reg[3] = (data >> 8)&0xff;
                tx_reg[4] = (data >> 16)&0xff;
                tx_reg[5] = (data >> 24)&0xff;
                write_reg(0x3E, tx_reg, 2);
                delay_1ms(1);
                tx_buffer[0] = checksum_calculate(&tx_reg[2], 4);
                tx_buffer[1] = 0x84; //combined length of register address and data
        write_reg(0x60, tx_buffer, 2); // Write the checksum and length
                delay_1ms(1);
              write_reg(0x40, &tx_reg[2], 4); // Write the checksum and length
                delay_1ms(1);
            break;
    }
}
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

5.4 RAM 寄存器读写函数

读写 RAM 寄存器强烈建议先进入 CONFIG_UPDATE 模式，完成后执行退出 CONFIG_UPDATE 模式的命令，配置开始生效。读写 RAM 寄存器将寄存器地址写入 0x3E，然后从 0x40 的数据缓冲区开始读取，以完成 RAM 寄存器的读取。具体的寄存器地址查阅 GD30BM2016 Register Maps 或者 GD30BM2016 的头文件。

I2C 写（注意 checksum 和 len 都只针对需要读写的数据）

S	SLV_ADDR	W	A	0x3E	A	addr_l	A	addr_h	A	CRC8	N	P	
S	SLV_ADDR	W	A	0x60	A	checksum	A	len(W)	A	CRC8	N	P	
S	SLV_ADDR	W	A	0x40	A	BYTE0	A	...	A	BYTEn	A	CRC8	N	P

I2C 读

S	SLV_ADDR	W	A	0x3E	A	addr_l	A	addr_h	A	CRC8	N	P
S	SLV_ADDR	W	A	0x61	A	len(R)	A	CRC8	A	P		

S	SLV_ADDR	W	A	0x40	A	RS	SLV_ADDR	R	A	BYTE0	A	...	BYTEn	A	CRC8	N	P
S	SLV_ADDR	W	A	0x60	A	RS	SLV_ADDR	R	A	BYTE0	A	...	BYTEn	A	CRC8	N	P

写 RAM 寄存器代码示例

```
/*!
    \brief        write gd30bm2016 register data
    \param[in]    reg_addr: register address
                  reg_data: register data
                  len: write data length
    \param[out] none
    \retval       none
*/
void bm2016_write_register(uint16_t reg_addr, uint32_t reg_data, uint8_t datalen)
{
    uint8_t tx_buffer[2] = {0x00, 0x00};
    uint8_t tx_regdata[6] = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00};

    //TX_RegData in little endian format
    tx_regdata[0] = reg_addr & 0xff;
    tx_regdata[1] = (reg_addr >> 8) & 0xff;
    tx_regdata[2] = reg_data & 0xff; //1st byte of data

    switch(datalen)
  {
        case 1: //1 byte datalength
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
                write_reg(0x3E, tx_regdata, 2);
                delay_1ms(1);
                tx_buffer[0] = checksum_calculate(&tx_regdata[2], 1);
                tx_buffer[1] = 0x81; //combined length of register address and data
                write_reg(0x60, tx_buffer, 2); // Write the checksum and length
                delay_1ms(1);
                write_reg(0x40, &tx_regdata[2], 1); // Write the checksum and length
                delay_1ms(1);
                break;
            case 2: //2 byte datalength
                tx_regdata[3] = (reg_data >> 8) & 0xff;
                write_reg(0x3E, tx_regdata, 2);
                delay_1ms(1);
                tx_buffer[0] = checksum_calculate(&tx_regdata[2], 2);
                tx_buffer[1] = 0x82; //combined length of register address and data
                write_reg(0x60, tx_buffer, 2); // Write the checksum and length
                delay_1ms(1);
              write_reg(0x40, &tx_regdata[2], 2); // Write the checksum and length
                delay_1ms(1);
                break;
            case 4: //4 byte datalength, Only used for CCGain and Capacity Gain
                tx_regdata[3] = (reg_data >> 8) & 0xff;
                tx_regdata[4] = (reg_data >> 16) & 0xff;
                tx_regdata[5] = (reg_data >> 24) & 0xff;
                write_reg(0x3E, tx_regdata, 2);
                delay_1ms(1);
                tx_buffer[0] = checksum_calculate(&tx_regdata[2], 4);
                tx_buffer[1] = 0x84; //combined length of register address and data
                write_reg(0x60, tx_buffer, 2); // Write the checksum and length
                delay_1ms(1);
                write_reg(0x40, &tx_regdata[2], 4); // Write the checksum and length
                delay_1ms(1);
                break;
    }
}
```

读 RAM 代码示例

```
/*!
    \brief          read gd30bm2016 register data
    \param[in]    reg_addr: register address
                    datalen: read data length
    \param[out] none
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
    \retval       none
*/
void bm2016_read_register(uint16_t reg_addr, uint8_t datalen)
{
    uint8_t tx_buffer[2] = {0x00, 0x00};
    uint8_t tx_regdata[6] = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00};

    //TX_RegData in little endian format
    tx_regdata[0] = reg_addr & 0xff;
    tx_regdata[1] = (reg_addr >> 8) & 0xff;
    write_reg(0x3E, tx_regdata, 2);
    delay_1ms(1);
    tx_buffer[0] = datalen; //combined length of register address and data
    write_reg(0x61, tx_buffer, 1); // Write the checksum and length
    delay_1ms(1);
    read_reg(0x40, datalen); // Write the checksum and length
    delay_1ms(1);
}
```

5.5 CRC 校验

CRC 多项式为 x8 + x2 + x + 1，初始值为 0。当器件到错误的 CRC 时，GD30BM2016 禁止更新对应寄存器的值。

CRC 代码示例

```
/*!
    \brief        crc calculates
    \param[in]    ptr: crc data
                  len: crc length
    \param[out] none
    \retval       crc result
*/
unsigned char crc8_calculate(unsigned char *ptr, unsigned char len)
{
    unsigned char i;
    unsigned char crc=0;
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
    while(len--!=0)
    {
        for(i=0x80; i!=0; i/=2){
            if((crc & 0x80) != 0)
            {
                crc *= 2;
                crc ^= 0x107;
            }
            else
                crc *= 2;
            if((*ptr & i)!=0)
                crc ^= 0x107;
        }
        ptr++;
    }
    return(crc);
}
```

5.6 CheckSum 校验

Checksum 计算方法为累加块地址寄存器中写入的所有数据，计算得到数据和后，将高 8 位右移后再与第八位相加，循环该步骤直到结果小于等于 0xFF，再用 0xFF 减去该结果最终所得为 checksum 校验值。

```
/*!
    \brief        checksum calculates
    \param[in]    ptr: checksum data
                  len: checksum length
    \param[out] none
    \retval       checksum result
*/
unsigned char checksum_calculate(unsigned char *ptr, unsigned char len)
{
    unsigned char i=0;
    uint32_t checksum = 0;

    while(i<len){
        checksum =checksum + ptr[i];
        i++;
    }
    while (checksum>0xff){
        checksum = (checksum >> 8) + (checksum & 0x000000ff);
    }
    checksum = 0xff - checksum;
    return(checksum);
}
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

6 命令和寄存器宏定义

为了快速配置使用 GD30BM2016 器件，该文档整理了直接命令、子文件命令、RAM 寄存器的宏定义头文件，用户可直接添加 GD30BM2016 宏定义至工程中，在通信函数调用接口直接通过宏定义映射对应的命令或者寄存器地址。下面是采用宏定义进入 FullAccessMode 的示例，其中 SEAL 对应的子命令为 0x032C，AccessKeyStep1 对应的子命令为 0x0280。

进入 FullAccessMode 示例

```
void Enter_FullAccessMode(void)
{
    CommandSubcommands(SEAL);
    GD30BM2016_SetRegister(AccessKeyStep1, 0x5A5A5A5A, 4);
    GD30BM2016_SetRegister(AccessKeyStep1, 0xA5A5A5A5, 4);
}
```

6.1 GD30BM2016 头文件

下面是常用直接命令、子文件命令、RAM 寄存器的宏定义，详细的定义请查阅 GD30BM2016 Register Maps 和 GD30BM2016 软件用户指南。

/*GD30BM2016 direct commands*/

```
#define ControlStatus_BMS          0x00
#define SafetyAlertA               0x02
#define SafetyStatusA              0x03
#define SafetyAlertB               0x04
#define SafetyStatusB              0x05
#define SafetyAlertC               0x06
#define SafetyStatusC              0x07
#define PFAlertA                   0x0A
#define PFStatusA                  0x0B
#define PFAlertB                   0x0C
#define PFStatusB                  0x0D
#define PFAlertC                   0x0E
#define PFStatusC                  0x0F
#define PFAlertD                   0x10
#define PFStatusD                  0x11
#define BatteryStatus              0x12
#define Cell1Voltage               0x14
#define Cell2Voltage               0x16
#define Cell3Voltage               0x18
#define Cell4Voltage               0x1A
#define Cell5Voltage               0x1C
#define Cell6Voltage               0x1E
#define Cell7Voltage               0x20
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define Cell8Voltage                0x22
#define Cell9Voltage                0x24
#define Cell10Voltage               0x28
#define Cell11Voltage               0x2A
#define Cell12Voltage               0x2C
#define Cell13Voltage               0x2E
#define Cell14Voltage               0x30
#define Cell15Voltage               0x32
#define Cell16Voltage               0x3C
#define StackVoltage                0x34
#define PACKPinVoltage              0x36
#define LDPinVoltage                0x38
#define CC2Current                  0x3A
#define AlarmStatus                 0x62
#define AlarmRawStatus              0x64
#define AlarmEnable                 0x66
#define IntTemperature              0x68
#define CFETOFFTemperature          0x6A
#define DFETOFFTemperature          0x6C
#define ALERTTemperature            0x6E
#define TS1Temperature              0x70
#define TS2Temperature              0x72
#define TS3Temperature              0x74
#define HDQTemperature              0x76
#define DCHGTemperature             0x78
#define DDSGTemperature             0x7A
#define FETStatus                   0x7F
```

/* GD30BM2016 Only Subcommands */

```
#define EXIT_DEEPSLEEP              0x0300
#define DEEPSLEEP                   0x0304
#define SHUTDOWN                    0x0308
#define GD30BM2016_RESET            0x030C
#define PDSGTEST                    0x0310
#define FUSE_TOGGLE                 0x0314
#define PCHGTEST                    0x0318
#define CHGTEST                     0x031C
#define DSGTEST                     0x0320
#define FET_ENABLE                  0x0324
#define PF_ENABLE                   0x0328
#define PF_RESET                    0x0029
#define SEAL                        0x032C
#define RESET_PASSQ                 0x0330
#define PTO_RECOVER                 0x0334
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define SET_CFGUPDATE                0x0338
#define EXIT_CFGUPDATE               0x033C
#define DSG_PDSG_OFF                 0x0340
#define CHG_PCHG_OFF                 0x0344
#define ALL_FETS_OFF                 0x0348
#define ALL_FETS_ON                  0x034C
#define SLEEP_ENABLE                 0x0350
#define SLEEP_DISABLE                0x0354
#define OCDL_RECOVER                 0x0358
#define SCDL_RECOVER                 0x035C
#define LOAD_DETECT_RESTART          0x0360
#define LOAD_DETECT_ON               0x0364
#define LOAD_DETECT_OFF              0x0368
#define CFETOFF_LO                   0x036C
#define DFETOFF_LO                   0x0370
#define ALERT_LO                     0x0374
#define HDQ_LO                       0x0378
#define DCHG_LO                      0x037C
#define DDSG_LO                      0x0380
#define CFETOFF_HI                   0x0384
#define DFETOFF_HI                   0x0388
#define ALERT_HI                     0x038C
#define HDQ_HI                       0x0390
#define DCHG_HI                      0x0394
#define DDSG_HI                      0x0398
#define PF_FORCE_A                   0x039C
#define PF_FORCE_B                   0x03A0
#define SWAP_COMM_MODE               0x03A4
#define SWAP_TO_I2C                  0x03A8
#define SWAP_TO_SPI                  0x03AC
#define SLEEP                        0x03B4
#define SPICS_LO                     0x03C0
#define SPICS_HI                     0x03C4
#define CHIP_RESET                   0x03CC
```

/* GD30BM2016 Subcommands with data */

```
#define DEVICE_NUMBER                0x0000
#define HW_VERSION                   0x000C
#define SyscState                    0x0010
#define OtpState                     0x0012
#define AdcSteta                     0x0013
#define PtState                      0x0014
#define CbState                      0x0015
#define Unseal_KeyStep1              0x0020
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define Unseal_KeyStep2                 0x0022
#define Full_Acess_KeyStep1             0x0024
#define Full_Acess_KeyStep2             0x0026
#define SAVED_PF_STATUS_A               0x0028
#define SAVED_PF_STATUS_B               0x0029
#define SAVED_PF_STATUS_C               0x002A
#define SAVED_PF_STATUS_D               0x002B
#define FuseFlag                        0x002C
#define MANUFACTURINGSTATUS             0x0030
#define Cell1_VolCount                  0x0080
#define Cell1_CurCount                  0x0084
#define Cell2_VolCount                  0x0088
#define Cell2_CurCount                  0x008C
#define Cell3_VolCount                  0x0090
#define Cell3_CurCount                  0x0094
#define Cell4_VolCount                  0x0098
#define Cell4_CurCount                  0x009C
#define Cell5_VolCount                  0x00A0
#define Cell5_CurCount                  0x00A4
#define Cell6_VolCount                  0x00AC
#define Cell6_CurCount                  0x00AC
#define Cell7_VolCount                  0x00B0
#define Cell7_CurCount                  0x00B4
#define Cell8_VolCount                  0x00B8
#define Cell8_CurCount                  0x00BC
#define Cell9_VolCount                  0x00C0
#define Cell9_CurCount                  0x00C4
#define Cell10_VolCount                 0x00D0
#define Cell10_CurCount                 0x00D4
#define Cell11_VolCount                 0x00D8
#define Cell11_CurCount                 0x00DC
#define Cell12_VolCount                 0x00E0
#define Cell12_CurCount                 0x00E4
#define Cell13_VolCount                 0x00E8
#define Cell13_CurCount                 0x00EC
#define Cell14_VolCount                 0x00F0
#define Cell14_CurCount                 0x00F4
#define Cell15_VolCount                 0x00F8
#define Cell15_CurCount                 0x00FC
#define VREG18                          0x0100
#define VSS                             0x0102
#define MaxCellVoltage                  0x0104
#define MinCellVoltage                  0x0106
#define BatteryVoltageSum               0x0108
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define CellTemp                    0x010A
#define FetTemp                     0x010C
#define MaxCellTemp_cmd             0x010E
#define MinCellTemp_cmd             0x0110
#define CC3Current                  0x0114
#define CC1Current                  0x0116
#define CC2Counts                   0x0118
#define CC3Counts                   0x011C
#define AccumCharge                 0x0120
#define AccumChargeFraction         0x0124
#define AccumTime                   0x0128
#define CFETOFFCounts               0x012C
#define DFETOFFCounts               0x0130
#define AlertCounts                 0x0134
#define TS1Counts                   0x0138
#define TS2Counts                   0x013C
#define TS3Counts                   0x0140
#define HDQCounts                   0x0144
#define DCHGCounts                  0x0148
#define DDSGounts                   0x014C
#define Cell16Current_Counts        0x0150
#define Cell16Voltage_Counts        0x0154
#define Cell16Voltage_CUV           0x0158
#define Cell16Voltage_COV           0x015A
#define Cell16TotalBalancingTime    0x015C
#define Cell1Voltage_CUV            0x0180
#define Cell2Voltage_CUV            0x0182
#define Cell3Voltage_CUV            0x0184
#define Cell4Voltage_CUV            0x0186
#define Cell5Voltage_CUV            0x0188
#define Cell6Voltage_CUV            0x018A
#define Cell7Voltage_CUV            0x018C
#define Cell8Voltage_CUV            0x018E
#define Cell9Voltage_CUV            0x0190
#define Cell10Voltage_CUV           0x0194
#define Cell11Voltage_CUV           0x0196
#define Cell12Voltage_CUV           0x0198
#define Cell13Voltage_CUV           0x019A
#define Cell14Voltage_CUV           0x019C
#define Cell15Voltage_CUV           0x019E
#define Cell1Voltage_COV            0x01A0
#define Cell2Voltage_COV            0x01A2
#define Cell3Voltage_COV            0x01A4
#define Cell4Voltage_COV            0x01A6
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define Cell5Voltage_COV                0x01A8
#define Cell6Voltage_COV                0x01AA
#define Cell7Voltage_COV                0x01AC
#define Cell8Voltage_COV                0x01AE
#define Cell9Voltage_COV                0x01B0
#define Cell10Voltage_COV               0x01B4
#define Cell11Voltage_COV               0x01B6
#define Cell12Voltage_COV               0x01B8
#define Cell13Voltage_COV               0x01BA
#define Cell14Voltage_COV               0x01BC
#define Cell15Voltage_COV               0x01BE
#define CellBalancingActiveCells        0x0200
#define CellBalancingSetLevel           0x0204
#define CellBalancingPresentTime        0x0208
#define Cell1TotalBalancingTime         0x0210
#define Cell2TotalBalancingTime         0x0214
#define Cell3TotalBalancingTime         0x0218
#define Cell4TotalBalancingTime         0x021C
#define Cell5TotalBalancingTime         0x0220
#define Cell6TotalBalancingTime         0x0224
#define Cell7TotalBalancingTime         0x0228
#define Cell8TotalBalancingTime         0x022C
#define Cell9TotalBalancingTime         0x0230
#define Cell10TotalBalancingTime        0x0238
#define Cell11TotalBalancingTime        0x023C
#define Cell12TotalBalancingTime        0x0240
#define Cell13TotalBalancingTime        0x0244
#define Cell14TotalBalancingTime        0x0248
#define Cell15TotalBalancingTime        0x024C
#define FETControl                      0x0250
#define REG12Control                    0x0254
#define OTPWriteCheckDataFailAddr       0x0258
#define OTPWriteCheckResult             0x025A
#define OTPWriteDataFailAddr            0x025C
#define OTPWriteResult                  0x025E
#define CC2AvgCounts                    0x0260
#define PACKpinADCCounts                0x0268
#define TopofStackADCCounts             0x026A
#define LDpinADCCounts                  0x026C
#define AccessKeyStep1                  0x0280
#define AccessKeyStep2                  0x0282
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

/* GD30BM2016 system registers */

```
#define PowerConfig                 0x8000    //Settings:Configuration:Power Config
#define REG12Config                 0x8002    //Settings:Configuration:REG12 Config
#define REG0Config                  0x8003    //Settings:Configuration:REG0 Config
#define HWDRegulatorOptions         0x8004    //Settings:Configuration:HWD Regulator Options
#define CommType                    0x8005    //Settings:Configuration:Comm Type
#define I2CAddress                  0x8006    //Settings:Configuration:I2C Address
#define SPIConfiguration            0x8007    //Settings:Configuration:SPI Configuration
#define CommIdleTime                0x8008    //Settings:Configuration:Comm Idle Time
#define LowVShutdownDelay           0x8009    //Power:Shutdown:Low V Shutdown Delay
#define ShutdownCellVoltage         0x800A    //Power:Shutdown:Shutdown Cell Voltage
#define ShutdownStackVoltage        0x800C    //Power:Shutdown:Shutdown Stack Voltage
#define ShutdownTemperature         0x800E    //Power:Shutdown:Shutdown Temperature
#define ShutdownTemperatureDelay    0x8010    //Power:Shutdown:Shutdown Temperature Delay
#define ShutdownCommandDelay        0x8011    //Power:Shutdown:Shutdown Command Delay
#define AutoShutdownTime            0x8012    //Power:Shutdown:Auto Shutdown Time
#define MANUFACTURING               0x8013    // PF_EN and FET_EN Configuration
#define SleepCurrent                0x8014    //Power:Sleep:Sleep Current
#define SleepHysteresisTime         0x8016    //Power:Sleep:Sleep Hysteresis Time
#define VoltageTime                 0x8017    //Power:Sleep:Voltage Time
#define WakeComparatorCurrent       0x8018    //Power:Sleep:Wake Comparator Current
#define SleepChgVoltageThreshold    0x801A    //Power:Sleep:Sleep Charger Voltage Threshold
#define SleepChgPACKTOSDelta        0x801C    //Power:Sleep:Sleep Charger PACK-TOS Delta
#define Cell1Interconnect           0x801E    //Settings:Interconnect Resistances:Cell 1 Interconnect
#define Cell2Interconnect           0x8020    //Settings:Interconnect Resistances:Cell 2 Interconnect
#define Cell3Interconnect           0x8022    //Settings:Interconnect Resistances:Cell 3 Interconnect
#define Cell4Interconnect           0x8024    //Settings:Interconnect Resistances:Cell 4 Interconnect
#define Cell5Interconnect           0x8026    //Settings:Interconnect Resistances:Cell 5 Interconnect
#define Cell6Interconnect           0x8028    //Settings:Interconnect Resistances:Cell 6 Interconnect
#define Cell7Interconnect           0x802A    //Settings:Interconnect Resistances:Cell 7 Interconnect
#define Cell8Interconnect           0x802C    //Settings:Interconnect Resistances:Cell 8 Interconnect
#define Cell9Interconnect           0x802E    //Settings:Interconnect Resistances:Cell 9 Interconnect
#define Cell10Interconnect          0x8032    //Settings:Interconnect Resistances:Cell 10 Interconnect
#define Cell11Interconnect          0x8034    //Settings:Interconnect Resistances:Cell 11 Interconnect
#define Cell12Interconnect          0x8036    //Settings:Interconnect Resistances:Cell 12 Interconnect
#define Cell13Interconnect          0x8038    //Settings:Interconnect Resistances:Cell 13 Interconnect
#define Cell14Interconnect          0x803A    //Settings:Interconnect Resistances:Cell 14 Interconnect
#define Cell15Interconnect          0x803C    //Settings:Interconnect Resistances:Cell 15 Interconnect
#define Cell16Interconnect          0x803E    //Settings:Interconnect Resistances:Cell 16 Interconnect
#define UnsealKeyStep1              0x8040    //Security:Keys:Unseal Key Step 1
#define UnsealKeyStep2              0x8042    //Security:Keys:Unseal Key Step 2
#define FullAccessKeyStep1          0x8044    //Security:Keys:Full Access Key Step 1
#define FullAccessKeyStep2          0x8046    //Security:Keys:Full Access Key Step 2
#define SecuritySettings            0x8048    //Security:Settings:Security Settings
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define ADC_CLK_CFG              0x8049
#define OTP_CTL                  0x804A
#define WDG_EN                   0x804C
#define FETOffDelay              0x804D      //Power:Shutdown:FET Off Delay
#define SCAN_MODE                0x804E

/* GD30BM2016 pad registers */
#define CFETOFFPinConfig         0x8200      //Settings:Configuration:CFETOFF Pin Config
#define DFETOFFPinConfig         0x8201      //Settings:Configuration:DFETOFF Pin Config
#define ALERTPinConfig           0x8202      //Settings:Configuration:ALERT Pin Config
#define SPICSPinConfig           0x8203      //Settings:Configuration:TS1 Config
#define SCLPinConfig             0x8204      //Settings:Configuration:TS2 Config
#define SDAPinConfig             0x8205      //Settings:Configuration:TS3 Config
#define HDQPinConfig             0x8206      //Settings:Configuration:HDQ Pin Config
#define DCHGPinConfig            0x8207      //Settings:Configuration:DCHG Pin Config
#define DDSGPinConfig            0x8208      //Settings:Configuration:DDSG Pin Config
#define TS1Config                0x8209
#define PAD_CFG                  0x8210
#define TS23Config               0x820A

/* GD30BM2016 adc registers */
#define Cell1Gain                0x8400      //Calibration:Voltage:Cell 1 Gain
#define Cell2Gain                0x8402      //Calibration:Voltage:Cell 2 Gain
#define Cell3Gain                0x8404      //Calibration:Voltage:Cell 3 Gain
#define Cell4Gain                0x8406      //Calibration:Voltage:Cell 4 Gain
#define Cell5Gain                0x8408      //Calibration:Voltage:Cell 5 Gain
#define Cell6Gain                0x840A      //Calibration:Voltage:Cell 6 Gain
#define Cell7Gain                0x840C      //Calibration:Voltage:Cell 7 Gain
#define Cell8Gain                0x840E      //Calibration:Voltage:Cell 8 Gain
#define Cell9Gain                0x8410      //Calibration:Voltage:Cell 9 Gain
#define Cell10Gain               0x8414      //Calibration:Voltage:Cell 10 Gain
#define Cell11Gain               0x8416      //Calibration:Voltage:Cell 11 Gain
#define Cell12Gain               0x8418      //Calibration:Voltage:Cell 12 Gain
#define Cell13Gain               0x841A      //Calibration:Voltage:Cell 13 Gain
#define Cell14Gain               0x841C      //Calibration:Voltage:Cell 14 Gain
#define Cell15Gain               0x841E      //Calibration:Voltage:Cell 15 Gain
#define Cell16Gain               0x8420      //Calibration:Voltage:Cell 16 Gain
#define PackGain                 0x8422      //Calibration:Voltage:Pack Gain
#define TOSGain                  0x8424      //Calibration:Voltage:TOS Gain
#define LDGain                   0x8426      //Calibration:Voltage:LD Gain
#define ADCGain                  0x8428      //Calibration:Voltage:ADC Gain
#define CCGain                   0x842C      //Calibration:Current:CC Gain
#define CC2Gain                  0x8430      //Calibration:Current:CC2 Gain
#define CC2Offset                0x8432      //Calibration:Current:CC2 offset
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define VcellOffset                     0x8434      //Calibration:Vcell Offset:Vcell Offset
#define VdivOffset                      0x8436      //Calibration:V Divider Offset:Vdiv Offset
#define ADCOffset                       0x8438      //Calibration: ADCoffset
#define CoulombCounterOffsetSamples     0x843A      //Calibration:Current Offset:Coulomb Counter Offset Samples
#define BoardOffset                     0x843C      //Calibration:Current Offset:Board Offset
#define SleepModeOffset                 0x843E      //Calibration: SleepModeOffset
#define IntGain                         0x8440      //Calibration:Internal Temp Model:Int Gain
#define InternalTempOffset              0x8442      //Calibration:Internal Temp Model:IntTempOffset
#define IntMaximumAD                    0x8444      //Calibration:Internal Temp Model:Int Maximum AD
#define IntMaximumTemp                  0x8446      //Calibration:Internal Temp Model:Int Maximum Temp
#define T18kCoeffa1                     0x8448      //Calibration:18K Temperature Model:Coeff a1
#define T18kCoeffa2                     0x844A      //Calibration:18K Temperature Model:Coeff a2
#define T18kCoeffa3                     0x844C      //Calibration:18K Temperature Model:Coeff a3
#define T18kCoeffa4                     0x844E      //Calibration:18K Temperature Model:Coeff a4
#define T180kCoeffa1                    0x8450      //Calibration:180K Temperature Model:Coeff a1
#define T180kCoeffa2                    0x8452      //Calibration:180K Temperature Model:Coeff a2
#define T180kCoeffa3                    0x8454      //Calibration:180K Temperature Model:Coeff a3
#define T180kCoeffa4                    0x8456      //Calibration:180K Temperature Model:Coeff a4
#define CustomCoeffa1                   0x8458      //Calibration:Custom Temperature Model:Coeff a1
#define CustomCoeffa2                   0x845A      //Calibration:Custom Temperature Model:Coeff a2
#define CustomCoeffa3                   0x845C      //Calibration:Custom Temperature Model:Coeff a3
#define CustomCoeffa4                   0x845E      //Calibration:Custom Temperature Model:Coeff a4
#define CoulombCounterDeadband          0x8460      //Calibration:Current Deadband:Coulomb Counter Deadband
#define CC3Samples                      0x8461      //Settings:Configuration:CC3 Samples
#define ADC_CFG                         0x8462
#define ADC_CC1_SEL                     0x8463
#define ADC_CC2_RSENSE                  0x8464
#define Code_Gain                       0x8466
#define Code_Offset                     0x8468
#define ADC_EN                          0x846A
```

/* GD30BM2016 protect registers */

```
#define SFCtrl                          0x8800      //Calibration:SFCtrl
#define LoadRemoveTH                    0x8802      //Calibration:LoadRemoveTH
#define MinBlowFuseVoltage              0x8804      //Settings:Fuse:Min Blow Fuse Voltage
#define FuseBlowTimeout                 0x8806      //Settings:Fuse:Fuse Blow Timeout
#define DAConfig                        0x8807      //Settings:Configuration:DA Configuration
#define VCellMode                       0x8808      //Settings:Configuration:Vcell Mode
#define ProtectionConfiguration         0x880A      //Settings:Protection:Protection Configuration
#define EnabledProtectionsA             0x880C      //Settings:Protection:Enabled Protections A
#define EnabledProtectionsB             0x880D      //Settings:Protection:Enabled Protections B
#define EnabledProtectionsC             0x880E      //Settings:Protection:Enabled Protections C
#define CHGFETProtectionsA              0x880F      //Settings:Protection:CHG FET Protections A
#define CHGFETProtectionsB              0x8810      //Settings:Protection:CHG FET Protections B
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define CHGFETProtectionsC          0x8811      //Settings:Protection:CHG FET Protections C
#define DSGFETProtectionsA          0x8812      //Settings:Protection:DSG FET Protections A
#define DSGFETProtectionsB          0x8813      //Settings:Protection:DSG FET Protections B
#define DSGFETProtectionsC          0x8814      //Settings:Protection:DSG FET Protections C
#define BodyDiodeThreshold          0x881C      //Settings:Protection:Body Diode Threshold
#define DefaultAlarmMask            0x881E      //Settings:Alarm:Default Alarm Mask
#define SFAlertMaskA                0x8815      //Settings:Alarm:SF Alert Mask A
#define SFAlertMaskB                0x8816      //Settings:Alarm:SF Alert Mask B
#define SFAlertMaskC                0x8817      //Settings:Alarm:SF Alert Mask C
#define PFAlertMaskA                0x8818      //Settings:Alarm:PF Alert Mask A
#define PFAlertMaskB                0x8819      //Settings:Alarm:PF Alert Mask B
#define PFAlertMaskC                0x881A      //Settings:Alarm:PF Alert Mask C
#define PFAlertMaskD                0x881B      //Settings:Alarm:PF Alert Mask D
#define EnabledPFA                  0x8820      //Settings:Permanent Failure:Enabled PF A
#define EnabledPFB                  0x8821      //Settings:Permanent Failure:Enabled PF B
#define EnabledPFC                  0x8822      //Settings:Permanent Failure:Enabled PF C
#define EnabledPFD                  0x8823      //Settings:Permanent Failure:Enabled PF D
#define FETOptions                  0x8824      //Settings:FET:FET Options
#define ChgPumpControl              0x8825      //Settings:FET:Chg Pump Control
#define PrechargeStartVoltage       0x8826      //Settings:FET:Precharge Start Voltage
#define PrechargeStopVoltage        0x8828      //Settings:FET:Precharge Stop Voltage
#define PredischargeTimeout         0x882A      //Settings:FET:Predischarge Timeout
#define PredischargeStopDelta       0x882B      //Settings:FET:Predischarge Stop Delta
#define CUVThreshold                0x882C      //Protections:CUV:Threshold
#define CUVDelay                                0x882E      //Protections:CUV:Delay
#define CUVRecoveryHysteresis       0x8830      //Protections:CUV:Recovery Hysteresis
#define COVThreshold                0x8832      //Protections:COV:Threshold
#define COVDelay                    0x8834      //Protections:COV:Delay
#define COVRecoveryHysteresis       0x8836      //Protections:COV:Recovery Hysteresis
#define COVLLatchLimit              0x8837      //Protections:COVL:Latch Limit
#define COVLCounterDecDelay         0x8838      //Protections:COVL:Counter Dec Delay
#define COVLRecoveryTime            0x8839      //Protections:COVL:Recovery Time
#define OCCThreshold                0x883A      //Protections:OCC:Threshold
#define OCCDelay                    0x883C      //Protections:OCC:Delay
#define OCCRecoveryThreshold        0x8840      //Protections:OCC:Recovery Threshold
#define OCCPACKTOSDelta             0x8842      //Protections:OCC:PACK-TOS Delta
#define OCD1Threshold               0x8844      //Protections:OCD1:Threshold
#define OCD1Delay                   0x8846      //Protections:OCD1:Delay
#define OCD2Threshold               0x8848      //Protections:OCD2:Threshold
#define OCD2Delay                   0x884A      //Protections:OCD2:Delay
#define SCDThreshold                0x884B      //Protections:SCD:Threshold
#define SCDDelay                    0x884C      //Protections:SCD:Delay
#define SCDRecoveryTime             0x884D      //Protections:SCD:Recovery Time
#define OCD3Threshold               0x884E      //Protections:OCD3:Threshold
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define OCD3Delay                    0x8850    //Protections:OCD3:Delay
#define OCDRecoveryThreshold         0x8852    //Protections:OCD:Recovery Threshold
#define OCDLLatchLimit               0x8854    //Protections:OCDL:Latch Limit
#define OCDLCounterDecDelay          0x8855    //Protections:OCDL:Counter Dec Delay
#define OCDLRecoveryTime             0x8856    //Protections:OCDL:Recovery Time
#define OCDLRecoveryThreshold        0x8858    //Protections:OCDL:Recovery Threshold
#define SCDLLatchLimit               0x885A    //Protections:SCDL:Latch Limit
#define SCDLCounterDecDelay          0x885B    //Protections:SCDL:Counter Dec Delay
#define SCDLRecoveryTime             0x885C    //Protections:SCDL:Recovery Time
#define SCDLRecoveryThreshold        0x885E    //Protections:SCDL:Recovery Threshold
#define OTCThreshold                 0x8860    //Protections:OTC:Threshold
#define OTCDelay                     0x8861    //Protections:OTC:Delay
#define OTCRecovery                  0x8862    //Protections:OTC:Recovery
#define OTDThreshold                 0x8863    //Protections:OTD:Threshold
#define OTDDelay                     0x8864    //Protections:OTD:Delay
#define OTDRecovery                  0x8865    //Protections:OTD:Recovery
#define OTFThreshold                 0x8866    //Protections:OTF:Threshold
#define OTFDelay                     0x8867    //Protections:OTF:Delay
#define OTFRecovery                  0x8868    //Protections:OTF:Recovery
#define OTINTThreshold               0x8869    //Protections:OTINT:Threshold
#define OTINTDelay                   0x886A    //Protections:OTINT:Delay
#define OTINTRecovery                0x886B    //Protections:OTINT:Recovery
#define UTCThreshold                 0x886C    //Protections:UTC:Threshold
#define UTCDelay                     0x886D    //Protections:UTC:Delay
#define UTCRecovery                  0x886E    //Protections:UTC:Recovery
#define UTDThreshold                 0x886F    //Protections:UTD:Threshold
#define UTDDelay                     0x8870    //Protections:UTD:Delay
#define UTDRecovery                  0x8871    //Protections:UTD:Recovery
#define UTINTThreshold               0x8872    //Protections:UTINT:Threshold
#define UTINTDelay                   0x8873    //Protections:UTINT:Delay
#define UTINTRecovery                0x8874    //Protections:UTINT:Recovery
#define ProtectionsRecoveryTime      0x8875    //Protections:Recovery:Time
#define HWDDelay                     0x8876    //Protections:HWD:Delay
#define PTOChargeThreshold           0x8878    //Protections:PTO:Charge Threshold
#define PTODelay                     0x887A    //Protections:PTO:Delay
#define PTOReset                     0x887C    //Protections:PTO:Reset
#define CUDEPThreshold               0x887E    //Permanent Fail:CUDEP:Threshold
#define CUDEPDelay                   0x8880    //Permanent Fail:CUDEP:Delay
#define SUVThreshold                 0x8882    //Permanent Fail:SUV:Threshold
#define SUVDelay                     0x8884    //Permanent Fail:SUV:Delay
#define SOVThreshold                 0x8886    //Permanent Fail:SOV:Threshold
#define SOVDelay                     0x8888    //Permanent Fail:SOV:Delay
#define TOSSThreshold                0x888A    //Permanent Fail:TOS:Threshold
#define TOSSDelay                    0x888C    //Permanent Fail:TOS:Delay
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define SOCCThreshold          0x888E    //Permanent Fail:SOCC:Threshold
#define SOCCDelay              0x8890    //Permanent Fail:SOCC:Delay
#define SOCDThreshold          0x8892    //Permanent Fail:SOCD:Threshold
#define SOCDDelay              0x8894    //Permanent Fail:SOCD:Delay
#define SOTThreshold           0x8895    //Permanent Fail:SOT:Threshold
#define SOTDelay               0x8896    //Permanent Fail:SOT:Delay
#define SOTFThreshold          0x8897    //Permanent Fail:SOTF:Threshold
#define SOTFDelay              0x8898    //Permanent Fail:SOTF:Delay
#define VIMRCheckVoltage       0x889A    //Permanent Fail:VIMR:Check Voltage
#define VIMRMaxRelaxCurrent    0x889C    //Permanent Fail:VIMR:Max Relax Current
#define VIMRThreshold          0x889E    //Permanent Fail:VIMR:Threshold
#define VIMRDelay              0x88A0    //Permanent Fail:VIMR:Delay
#define VIMRRelaxMinDuration   0x88A2    //Permanent Fail:VIMR:Relax Min Duration
#define VIMACheckVoltage       0x88A4    //Permanent Fail:VIMA:Check Voltage
#define VIMAMinActiveCurrent   0x88A6    //Permanent Fail:VIMA:Min Active Current
#define VIMAThreshold          0x88A8    //Permanent Fail:VIMA:Threshold
#define VIMADelay              0x88AA    //Permanent Fail:VIMA:Delay
#define CFETFOFFThreshold      0x88AC    //Permanent Fail:CFETF:OFF Threshold
#define CFETFOFFDelay          0x88AE    //Permanent Fail:CFETF:OFF Delay
#define DFETFOFFThreshold      0x88B0    //Permanent Fail:DFETF:OFF Threshold
#define DFETFOFFDelay          0x88B2    //Permanent Fail:DFETF:OFF Delay
#define VSSFFailThreshold      0x88B4    //Permanent Fail:VSSF:Fail Threshold
#define VSSFDelay              0x88B6    //Permanent Fail:VSSF:Delay
#define PF2LVLDelay            0x88B7    //Permanent Fail:2LVL:Delay
#define LFOFDelay              0x88B8    //Permanent Fail:LFOF:Delay
#define HWMXDelay              0x88BA    //Permanent Fail:HWMX:Delay
#define SFFaultMaskA           0x88BB
#define SFFaultMaskB           0x88BC
#define SFFaultMaskC           0x88BD
```

/* GD30BM2016 cell balance registers */

```
#define BalanceConfig                0x8C00    //Settings:Cell Balancing Config:Balancing Configuration
#define MinCellTemp                  0x8C01    //Settings:Cell Balancing Config:Min Cell Temp
#define MaxCellTemp                  0x8C02    //Settings:Cell Balancing Config:Max Cell Temp
#define MaxInternalTemp              0x8C03    //Settings:Cell Balancing Config:Max Internal Temp
#define CellBalanceInterval          0x8C04    //Settings:Cell Balancing Config:Cell Balance Interval
#define CellBalanceMaxCells          0x8C05    //Settings:Cell Balancing Config:Cell Balance Max Cells
#define CellBalanceMinCellVCharge    0x8C06    //Settings:Cell Balancing Config:Cell Balance Min Cell V (Charge)
#define CellBalanceMinDeltaCharge    0x8C08    //Settings:Cell Balancing Config:Cell Balance Min Delta (Charge)
#define CellBalanceStopDeltaCharge   0x8C09    //Settings:Cell Balancing Config:Cell Balance Stop Delta
#define CellBalanceMinCellVRelax     0x8C0A    //Settings:Cell Balancing Config:Cell Balance Min Cell V (Relax)
#define CellBalanceMinDeltaRelax     0x8C0C    //Settings:Cell Balancing Config:Cell Balance Min Delta (Relax
#define CellBalanceStopDeltaRelax    0x8C0D    //Settings:Cell Balancing Config:Cell Balance Stop Delta (Relax)
#define LoadDetectActiveTime         0x8C0E    //Protections:Load Detect:Active Time
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
#define LoadDetectRetryDelay    0x8C0F    //Protections:Load Detect:Retry Delay
#define LoadDetectTimeout       0x8C10    //Protections:Load Detect:Timeout
#define ChgCurrentThreshold     0x8C14    //Settings:Current Thresholds:Chg Current Threshold
#define CheckTime               0x8C16    //Settings:Cell Open-Wire:Check Time
#define DsgCurrentThreshold     0x8C12    //Settings:Current Thresholds:Dsg Current Threshold
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

7 历史版本

版本号	描述	日期
A	初始发布	2024 年 03 月 20 日
B	增加 BQ769142 和 BQ76942 移植描述	2024 年 08 月 10 日
C	增加 BM2016 常用功能解答章节	2025 年 6 月 10 日
D	修改电流配置、均衡等描述	2025 年 12 月 15 日
E	更新 COV、CUV 迟滞单位，更新 CC2 current 计算，增加电流读数描述章节	2026 年 3 月 17 日
F	更新差异对比：引脚、性能、寄存器等 更新常用功能配置章节：汇总功能模块用的寄存器； 增加上电时序章节，增加看门狗使用	2026 年 3 月 22 日

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Important Notice

This document is the property of GigaDevice Semiconductor Inc. and its subsidiaries (the "Company"). This document, including any product of the Company described in this document (the “Product”), is owned by the Company according to the laws of the People’s Republic of China and other applicable laws. The Company reserves all rights under such laws and no Intellectual Property Rights are transferred (either wholly or partially) or licensed by the Company (either expressly or impliedly) herein. The names and brands of third party referred thereto (if any) are the property of their respective owner and referred to for identification purposes only.

The Company makes no representations or warranties of any kind, express or implied, with regard to the merchantability and the fitness for a particular purpose of the Product, nor does the Company assume any liability arising out of the application or use of any Product described in this document. Any information provided in this document is provided only for reference purposes. It is the sole responsibility of the user of this document to determine whether the Product is suitable and fit for its applications and products planned, and properly design, program, and test the functionality and safety of its applications and products planned using the Product. Unless otherwise expressly specified in the datasheet of the Product , the Product is designed, developed, and/or manufactured for ordinary business, industrial, personal, and/or household applications only, and the Product is not designed or intended for use in (i) safety critical applications such as weapons systems, nuclear facilities, atomic energy controller, combustion controller, aeronautic or aerospace applications, traffic signal instruments, pollution control or hazardous substance management; (ii) life-support systems, other medical equipment or systems (including life support equipment and surgical implants); (iii) automotive applications or environments, including but not limited to applications for active and passive safety of automobiles (regardless of front market or aftermarket), for example, EPS, braking, ADAS (camera/fusion), EMS, TCU, BMS, BSG, TPMS, Airbag, Suspension, DMS, ICMS, Domain, ESC, DCDC, e-clutch, advanced-lighting, etc.. Automobile herein means a vehicle propelled by a self-contained motor, engine or the like, such as, without limitation, cars, trucks, motorcycles, electric cars, and other transportation devices; and/or (iv) other uses where the failure of the device or the Product can reasonably be expected to result in personal injury, death, or severe property or environmental damage (collectively "Unintended Uses"). Customers shall take any and all actions to ensure the Product meets the applicable laws and regulations. The Company is not liable for, in whole or in part, and customers shall hereby release the Company as well as its suppliers and/or distributors from, any claim, damage, or other liability arising from or related to all Unintended Uses of the Product. Customers shall indemnify and hold the Company, and its officers, employees, subsidiaries, affiliates as well as its suppliers and/or distributors harmless from and against all claims, costs, damages, and other liabilities, including claims for personal injury or death, arising from or related to any Unintended Uses of the Product.

Information in this document is provided solely in connection with the Product. The Company reserves the right to make changes, corrections, modifications or improvements to this document and the Product described herein at any time without notice. The Company shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them. Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2025 GigaDevice – All rights reserved

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.